

Shear integration of LKFs

Internship Report



Author: Anaëlle Havret
Program: SeaTech - MOCA 2A

Academic Advisor:
Mehmet Ersoy
University of Toulon

Training Supervisor:
Björn ERLINGSSON
University Centre of the Westfjords

Academic Year 2025–2026

Scientific and Academic integrity

Declaration of non-plagiarism

I, the undersigned, Anaëlle Havret
Student ID number : 22404770
Enrolled in 2A MOCA

Hereby declare that I have read the general academic regulations, and in particular section 7.3 concerning plagiarism. I am fully aware that copying in full, without citation or reference, any document or part of a document published in any form whatsoever (books, publications, student reports, internet, etc.) constitutes plagiarism and is a violation of copyright as well as a clear case of fraud. Consequently, I undertake to cite all the sources I used to produce and write this document

Use of AI

The purpose of this declaration is to promote transparency and develop your professional responsibility regarding digital tools. It aims to raise your awareness of the pedagogical implications of AI while allowing instructors to assess your ability to use it appropriately and responsibly.

☐ I did not use any AI tools to complete this report.



I used one or more AI tools to complete this report.

AI tool(s) used: Gemini



1 I used a generative AI tool to improve the writing (spelling, grammar, syntax) of the document I wrote.



2 I used an AI tool to structure my work and help me define the work plan.



3. I used an AI tool to translate texts included or not included in my work



4. I used a generative AI tool as a search engine to explore, simplify, or identify relevant references (without direct integration).



5 I used a generative AI tool to generate ideas or content, which I then reworked and integrated into my work.



6. If I used sentences entirely generated by AI, I cited them as a source.



7. I have kept the prompts and exchanges with the AI and will make them available to the examiner.

I certify that the use of AI in this work was carried out in accordance with the rules of academic integrity.

Done on 10/08/2026

Signing up



Résumé

Ce rapport de stage de recherche porte sur le comportement mécanique de la banquise arctique, et plus particulièrement sur l'analyse des fractures localisées qui la traversent, connues sous le nom de linéaments cinématiques de glace (LKFs, pour *Linear Kinematic Features*). L'objectif principal de ce travail est de reconstruire le champ de contraintes de cisaillement au sein de ces LKFs en se basant sur les forçages mécaniques externes (notamment le vent), la divergence locale du champ de déformation et l'analyse géométrique de la trajectoire des fractures.

En couplant les observations satellitaires du système RGPS (*RADARSAT Geophysical Processor System*) avec les données de réanalyse atmosphérique et océanique de Copernicus, cette étude met en œuvre la méthode des caractéristiques pour intégrer le cisaillement autour des géométries de LKFs observées. Une analyse numérique approfondie a permis d'optimiser l'algorithme d'intégration et d'évaluer sa sensibilité au pas de maillage spatial. Ce travail pourrait contribuer à une meilleure quantification des contraintes mécaniques à moyenne échelle en Arctique, offrant des applications directes pour la modélisation de la dérive de la glace et la sécurité des routes maritimes polaires.

Mots-clés : banquise, LKF, cisaillement, divergence, géométrie des fractures, méthode des caractéristiques, forçages externes, RGPS, Copernicus, intégration de contrainte, Arctique.

Abstract

This research internship report focuses on the mechanical behavior of Arctic sea ice, specifically investigating the localized fracture lines that disrupt the ice pack, known as Linear Kinematic Features (LKFs). The primary objective of this work is to reconstruct the internal shear stress field within these LKFs using external mechanical forcing parameters (primarily wind stress), local deformation divergence, and the observed geometric trajectory of the fracture lines.

By coupling satellite observations from the RADARSAT Geophysical Processor System (RGPS) with atmospheric and oceanic reanalysis fields from Copernicus, this study implements the Method of Characteristics to integrate shear stress around observed LKF geometries. Detailed numerical analyses were conducted to optimize the integration scheme and evaluate spatial grid sensitivity. This work could contribute to an enhanced quantification of mesoscale sea ice stresses, with direct operational applications for drift modeling and maritime route safety in polar regions.

Keywords: sea ice, LKF, shear stress, divergence, fracture geometry, method of characteristics, external forcing, RGPS, Copernicus, stress integration, Arctic.

Contents

1	Introduction	3
1.1	Context	3
1.2	State of the art	3
1.3	Internship objectives and scope	4
1.4	Report structure	4
2	The host institution: University centre of the Westfjords	5
3	Internship experience and core missions	5
3.1	Data Source Taxonomy	6
4	Theoretical foundations: continuum mechanics	7
4.1	Geometric representation and properties of Mohr's circle	7
4.2	Limit state analysis	8
4.3	Application to slip planes and failure criteria	8
4.4	2D symmetric stress tensor	8
4.5	Track 1: Diagnostic core audit of the neXtSIM model	10
4.6	Track 2: Direct copernicus inversion via brittle BBM rheology	10
4.7	Track 3: Shear integration via geometric curvature vs. characteristic formulations	10
5	Final track - Reconstruction of shear stress profiles: a joint characteristic and LKF geometry approach	11
5.1	Mathematical formulation and characteristic line integration	11
5.2	Initial interpretation: branching tree reconstruction algorithm	11
5.3	Method 2: Loop order inversion (no angle recalculation during the first iteration)	13
5.4	Algorithmic optimization: transition to dynamic convergence criteria	15
5.5	Mesh convergence analysis	15
5.5.1	Quantitative convergence analysis: log-log error scaling	16
5.5.2	Final Saturation Level of Shear Stress Z	17
5.6	Multi-level saturation of shear stress	19
5.7	Computation of a third characteristic S_3	23
5.8	Relation between $\Delta Z/Z$ vs ϕ	25
5.9	Relation between ΔL_i vs ϕ	26
5.10	LKF propagation simulation and wind stress integration	27
6	Conclusion	29

Acknowledgements

I would like to express my sincere gratitude to my internship supervisor, Björn Erlingsson, for giving me the opportunity to work on this research project on sea ice dynamics and for sharing his knowledge throughout this experience.

I would also like to thank my academic advisor from SeaTech, Mehmet Ersoy, for his continuous support throughout my internship.

My appreciation also goes to the University Centre of the Westfjords for welcoming me and providing a unique academic environment.

Finally, I would like to extend my thanks to my team members and colleagues for their collaboration, mutual support, and valuable discussions that helped me overcome theoretical challenges.

1 Introduction

1.1 Context

Sea ice is not a static cover: it drifts continuously under wind and ocean forcing, and even though drift speeds stay relatively low, the internal stresses building up inside the ice pack can exceed its yield threshold. When that happens, the ice fails abruptly — either through ridging, where blocks pile up chaotically, or rafting, where one sheet slides over another. These failure mechanisms are governed by the brittle behavior of polycrystalline ice, which is largely scale-independent [10]. The resulting deformation regimes combine compression and shear forces that trigger distinct failure modes, such as chaotic pressure ridge building (*ridging*) and sheet overthrusting (*rafting*).

These failures don't happen everywhere at once. Instead, deformation and stress concentrate along narrow, discontinuous lines called Linear Kinematic Features (LKFs), which follow a multifractal spatial pattern [6]. Once a fracture opens along an LKF, the ice floes on either side start drifting independently, colliding with neighboring floes and reshaping the ice cover.

To track these fractures from space, we rely on satellite radar systems such as the RADARSAT Geophysical Processor System (RGPS). These systems send microwave pulses toward the surface and record what bounces back. Because leads, thin ice, and ridged zones each reflect radar differently, this backscatter signal lets researchers trace individual fractures across large areas of the Arctic.

The idea behind this internship is to go one step further than simply mapping these fractures: by looking at the curvature and orientation of observed LKFs, we try to reconstruct the internal shear stress field that produced them. Beyond the theoretical interest, understanding these stress states has a direct practical use — predicting where and how the ice is likely to fail next matters for maritime safety and for planning shipping routes through polar corridors.

1.2 State of the art

To automate the mapping of structural features from space, modern remote sensing increasingly exploits machine learning algorithms. For instance, convolutional neural networks (CNNs) trained on high-resolution Sentinel-1 SAR data are now deployed to automatically classify distinct sea ice types and isolate structural fracture zones [14b].

To numerically represent these interactions, the Elastic-Viscous-Plastic (EVP) rheology has long served as the standard framework for large-scale ice dynamics calculations. However, classic continuous formulations struggle to resolve the localized, anisotropic patterns inherent to brittle failure. To address this limitation, the Brittle Bingham-Maxwell (BBM) rheology was developed [7], establishing a framework where progressive damage mechanics and viscoelastic flow models are integrated into a Lagrangian triangular mesh architecture.

Furthermore, accurate stress integration at the mesoscale requires moving beyond constant drag coefficients by introducing variable parameterizations that account for sea ice geometry and form drag [5]. The computational challenge of tuning these non-linear properties has recently found an automated path via machine learning optimization schemes applied to the core parameters of macroscopic ice simulations [4].

Finally, recent data analysis indicates that momentum transfer across the air-ice-ocean interface is evolving: the ongoing thinning of the Arctic ice cover has increased its mobility and altered its response to wind forcing, making the continuous analysis of drift trends essential for understanding the Arctic’s changing energy balance [9]. Tracking this evolving kinematic field relies on pure Lagrangian tools. Frameworks like OpenDrift simulate the direct dispersion of tracking elements [8], while advanced climate data records reconstruct historical ice age variations by executing the continuous Lagrangian advection of adaptive triangular meshes across the Arctic basin [1].

1.3 Internship objectives and scope

The internship is titled “Sea Ice Mesoscale Stress Integration”. The project targets the mechanical processing of Arctic Earth Observation Data accessible through the Copernicus Marine Service data space via the Copernicus Arctic-Hub.

This research is conducted in parallel by a team of three students focusing on complementary axes:

- **Axis 1 (My Focus):** Integrating shear stress profiles by coupling Copernicus atmospheric/oceanic reanalyses with RGPS satellite observations of LKFs, using characteristic equations along observed LKF geometries.
- **Axis 2:** Comparative benchmarking and algorithmic optimization of integration methods (Methods 1 and 2).
- **Axis 3:** Stochastic analysis of spatial fracture distributions and LKF networks.

The overarching scientific goal is to develop, implement, and benchmark computational pipelines to generate a high-resolution time series of mesoscale sea ice stress proxies (both mean pressure and shear stress) across the Arctic domain.

1.4 Report structure

The remainder of this report is organized as follows. Section 2 presents the host institution, the University Centre of the Westfjords. Section 3 details the overall internship environment, technical autonomy, and data source taxonomies. Section 4 establishes the theoretical framework of continuum mechanics, covering Mohr’s circle geometry, limit state analysis, slip-line curvature formulations, and hyperbolic characteristic equations. Finally, Section 5 delivers the core findings on shear stress reconstruction along LKF geometries, detailing the mathematical formulation, algorithmic optimization via dynamic convergence criteria, spatial mesh sensitivity, multi-level grid construction, and fracture chain integration (s_3).

2 The host institution: University centre of the Westfjords

The University Centre of the Westfjords (Háskóla­setur Vestfjarða, UW), established in 2005 and located in Ísafjörður, Iceland, is a specialized, interdisciplinary higher education and research hub. The institution offers postgraduate curricula—notably Master of Science degrees in Coastal and Marine Management as well as Coastal Communities and Regional Development. Its unique position in the sub-Arctic enables it to bridge localized coastal dynamics with global climatological and oceanographic operational structures. Through these programs, UW focuses on advancing academic innovation in high-latitude resource governance, driving the regional knowledge economy to mitigate geographic isolation, and operating as a field-station hub for Arctic research integration.

While operating alongside larger domestic institutions such as the University of Iceland (Háskóli Íslands) in Reykjavík and the University of Akureyri, UW maintains a distinct competitive advantage through its direct proximity to sub-Arctic marine corridors. Its primary ecosystem includes international graduate scholars, independent research scientists, and maritime operators seeking localized geographic expertise. To fulfill its scientific and environmental monitoring mission, the University Centre relies on strategic collaborations with key national and international organizations, including the Icelandic Meteorological Office (Veðurstofa Íslands) and the European Union’s Copernicus Marine Service (CMEMS).

3 Internship experience and core missions

The internship was conducted with a high degree of technical autonomy, characterized by self-directed exploration without predefined task lists or step-by-step guidance. Rather than following assigned operational duties, the direction of the project relied entirely on personal initiative, from defining research priorities to designing original methodologies. The core mission evolved from setting up a local Linux computing environment to conceiving and implementing new shear stress integration methods through continuous individual reflection and group brainstorming. Continuous academic research, self-directed progress tracking, and regular supervisory meetings structured the work throughout the project. Table 1 provides a synthetic summary of the operational domain, executed activities, and their respective time allocations.

Table 1: Synthetic summary of operational modules, executed tasks, and time allocation

Module / Domain	Technical Operations & Executed Tasks	Time Allocation
Python code Development	Design, implementation, and optimization of novel shear stress integration algorithms and scientific scripts.	50%
Environment & Config.	Manjaro Linux configuration on external SSD; SSH keys (ED25519); deployment of isolated <code>.venv</code> (Python, <code>xarray</code> , <code>cartopy</code> , <code>netCDF4</code>).	5%
Literature & Theory	Continuous literature review; study of BBM rheology; mathematical foundation of the Method of Characteristics.	10%
Individual & Group research	Independent reflection, theoretical problem-solving, active participation in collaborative discussions, and peer brainstorming.	10%
Meetings & Reporting	Daily progress logbook; preparation, presentation, and active participation in weekly supervisory and team meetings; report writing.	25%

3.1 Data Source Taxonomy

RGPS Data Satellite observations from the RADARSAT Geophysical Processor System (RGPS, 2001) were utilized. These datasets provide the most straightforward baseline for extracting fracture geometries, as each detected fracture is logged alongside its constituent nodal coordinates, shear strain, and spatial divergence.

Copernicus CMEMS data The spatial domain was constrained to the Fram Strait (17.0°W to 5.0°W / 76.0°N to 80.0°N). As a primary exit corridor for Arctic sea ice, this high-strain gateway exhibits extreme fracture density (LKFs) and ice-type heterogeneity. This spatial bounding minimizes numerical noise while ensuring high computational efficiency. To calculate shear stress accumulation, three hourly Copernicus Marine Service (CMEMS) datasets were ingested, harmonized, and interpolated onto a unified grid using Python (`xarray`):

- **Sea ice parameters [11]:** Ice concentration (`siconc`), ice thickness (`sithick`), sea surface height (`zos`), and drift velocity components (`vxsi`, `vysi`) used to derive strain rate tensors.
- **Ocean current hydrodynamics [11]:** Velocity fields (`uo`, `vo`) representing sub-ice oceanic drag forces.
- **Atmospheric forcing [12]:** Surface wind velocities and wind stress components driving aerodynamic ice fracturing.

neXtSIM model The **neXtSIM** (*Next-generation Sea Ice Model*), engineered by **NERSC**, is a high-resolution sea ice dynamic model designed to simulate ice mechanics and drift. Its core innovation relies on a mobile triangular Lagrangian mesh that tracks ice motion to eradicate numerical diffusion, coupled with the **Brittle Bingham-Maxwell (BBM)** rheology. This framework models sea ice as a brittle elastic solid, capturing the spontaneous formation and sharp localization of Linear Kinematic Features (LKF).

Daily deformation output files from NERSC yield two fundamental fields: divergence/convergence (e_{1_med}) and shear (e_{2_med}). These quantities characterize the sea ice mechanical state, including lead formation, ridge compaction, and lateral shearing. To maximize signal resolution, the scalar invariant of total strain ($\sqrt{e_1^2 + e_2^2}$) was computed, applying spatial threshold filtering and scale saturation to suppress background noise while accentuating LKF network contrast.

4 Theoretical foundations: continuum mechanics

To model the mechanical behavior of sea ice and analyze stress transmission within the ice cover, we rely on the principles of continuum mechanics. More specifically, this study adopts and adapts the theoretical formulations established by Björn Erlingsson, which lay the foundation for medium-scale sea ice stress states and rheology [14a; 13].

4.1 Geometric representation and properties of Mohr's circle

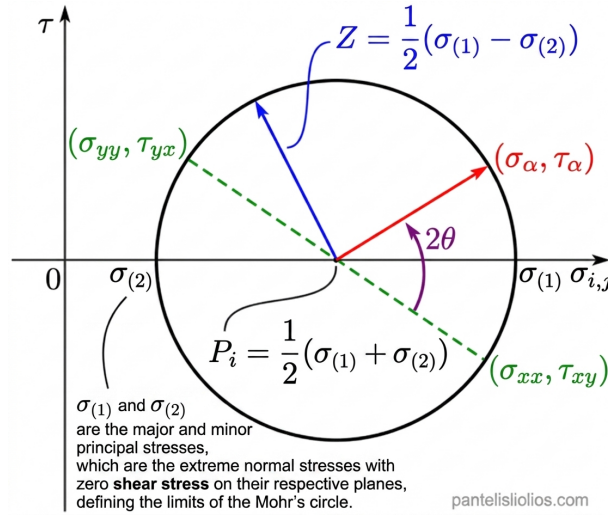


Figure 1: Schematic diagram of Mohr's circle

Mohr's circle is defined in a coordinate system where the horizontal axis (σ_n) represents normal stresses and the vertical axis (τ_n) represents shear stresses. To properly account for compressive states in sea ice, pressure is oriented along the negative direction of the horizontal axis. The stress variables translate directly into the geometric properties of the circle:

- **Circle center (Π):** Defined by the isotropic pressure (or two-dimensional mean pressure), corresponding to the average of the principal or normal stresses:

$$\Pi = \frac{\sigma_{xx} + \sigma_{yy}}{2} \quad \text{or} \quad \Pi = -\frac{\sigma_{(1)} + \sigma_{(2)}}{2}$$

- **Circle radius (Z):** Represents the maximum shear stress, computed from the normal

and shear components in cartesian coordinates:

$$Z = \sqrt{\left(\frac{\sigma_{xx} - \sigma_{yy}}{2}\right)^2 + \sigma_{xy}^2} = \frac{\sigma_{(1)} - \sigma_{(2)}}{2}$$

where $\sigma_{(1)}$ and $\sigma_{(2)}$ are the major and minor principal stresses, respectively. These correspond to the extreme normal stresses on planes where shear stress vanishes, thus defining the horizontal bounds of Mohr's circle.

According to the Coulomb failure criterion, macroscopic yield is reached along a fracture plane when the normal stress (σ_n) and shear stress (τ_n) on that plane touch the failure envelope. At any point on the circle, the stress components acting on a plane oriented at an angle 2θ relative to the principal stress axis are expressed as:

$$\sigma_n = \Pi + Z \cos(2\theta) \quad \text{and} \quad \tau_n = Z \sin(2\theta) \quad (1)$$

4.2 Limit state analysis

By evaluating the critical failure state within the stress invariant space defined by the maximum shear stress / radius (Z) and mean pressure (Π), the constitutive coupling is written as:

$$\frac{\partial Z}{\partial \Pi} = \sin(\phi) \quad (2)$$

This relationship expresses the pressure-dependency of the ice cover's shear strength as a function of the confinement pressure to which it is subjected.

4.3 Application to slip planes and failure criteria

The transformation from the global coordinate system to local slip planes is performed via tensor rotation. Following Björn Erlingsson's formulations [14a; 13], the directions of fracture planes (LKFs) form an angle λ with respect to the principal axis of compression. The orientations of these two conjugate fracture families, denoted $\theta_{1,2}$, are expressed as:

$$\theta_{1,2} = \beta \pm \lambda \quad \text{with} \quad \lambda = \frac{\pi}{4} + \frac{\phi}{2} \quad (3)$$

where ϕ is the internal friction angle of the sea ice matrix, and the orientation angle (β) designates the physical orientation angle of the major principal compressive stress relative to the geographical x-axis.

4.4 2D symmetric stress tensor

At a macroscopic scale, sea ice is modeled as a two-dimensional plastic medium under plane stress conditions. At any geographical coordinate (x, y) , the internal stress state is governed by the 2D symmetric stress tensor σ . Expressed in the fixed geographic coordinate frame using mean isotropic pressure Π and maximum shear stress Z , the tensor takes the following form:

$$\sigma = \begin{pmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{pmatrix} = \begin{pmatrix} -\Pi + Z \cos(2\beta) & Z \sin(2\beta) \\ Z \sin(2\beta) & -\Pi - Z \cos(2\beta) \end{pmatrix} \quad (4)$$

Method 0: The method of characteristics (slip line theory)

This method describes the spatial evolution of stress along the geometry of Linear Kinematic Features (LKFs). Along the characteristic slip lines S_1 and S_2 , the stress state satisfies:

$$\begin{aligned} \text{Along } S_1 : \quad & \frac{\partial \beta}{\partial S_1} - \frac{1}{2Z \tan(\phi)} \frac{\partial Z}{\partial S_1} = A_1 \\ \text{Along } S_2 : \quad & \frac{\partial \beta}{\partial S_2} + \frac{1}{2Z \tan(\phi)} \frac{\partial Z}{\partial S_2} = A_2 \end{aligned}$$

where A_1 and A_2 represent the geometric curvatures of the slip lines.

Method 1: Logarithmic stress tensor integration along characteristics

Integrating the equations above along a fracture with an angular change $\Delta\beta$ yields:

$$\frac{Z + \Delta Z}{Z} = \exp(\pm 2 \tan(\phi) \cdot \Delta\beta) \implies \frac{\Delta Z}{Z} = \exp(\pm 2 \tan(\phi) \cdot \Delta\beta) - 1 \quad (5)$$

This isolates purely geometric and structural stress variations driven by the directional steering of the fracture lines [14a](#).

Method 2: Analytical inversion of slip-line curvature (κ)

Method 2 extracts the geometric curvature $\kappa = \frac{d\beta}{ds}$ of localized LKFs across curvilinear arc-lengths s from satellite imagery (SAR/RGPS). Balancing spatial stress variations against the external atmospheric wind forcing vector $\vec{\tau}_{\text{wind}}$ along the characteristic path yields:

$$\kappa = \frac{f_{\text{wind}} \cdot \sin(\beta - \alpha_{\text{wind}} \pm \lambda)}{4Z} \implies Z = \frac{f_{\text{wind}} \cdot \sin(\beta - \alpha_{\text{wind}} \pm \lambda)}{4\kappa} \quad (6)$$

where f_{wind} and α_{wind} represent the magnitude and direction of the wind stress vector derived from Copernicus atmospheric reanalysis. Highly linear features ($\kappa \rightarrow 0$) imply high internal shear resistance Z , while highly curved features imply lower structural resistance.

Method 3: Kinematic variational principle & maximum energy dissipation

Method 3 utilizes sea ice velocity fields $\vec{v} = (u, v)$ derived from RGPS or Copernicus satellite tracking. Invoking the Principle of Maximum Energy Dissipation, the mechanical work rate per unit area Q is maximized under plastic flow constraints using a localized Lagrange multiplier λ :

$$Q = \sigma_{ij} \dot{\epsilon}_{ij} = \lambda \cdot \dot{\epsilon}_{II} \quad (7)$$

where $\dot{\epsilon}_{ij}$ is the 2D strain rate tensor. Formulating the associated flow rule for a plastic Mohr-Coulomb material under kinematic velocity divergence $\nabla \cdot \vec{v} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$ yields:

$$\dot{\epsilon} = \frac{\nabla \cdot \vec{v}}{2} \begin{pmatrix} 1 - \frac{\cos(2\beta)}{\sin(\phi)} & -\frac{\sin(2\beta)}{\sin(\phi)} \\ -\frac{\sin(2\beta)}{\sin(\phi)} & 1 + \frac{\cos(2\beta)}{\sin(\phi)} \end{pmatrix} \quad \text{with} \quad \frac{\dot{\epsilon}_{II}}{\dot{\epsilon}_I} = \frac{1}{\sin(\phi)} \quad (8)$$

Several approaches were initially explored to model the spatial evolution of internal shear stress.

4.5 Track 1: Diagnostic core audit of the neXtSIM model

To dissect the model's inner mechanics, the diagnostic codebase developed by Dr. Anton Korosov was audited. A dedicated `conda` kernel environment was instantiated within the execution notebook to resolve all dependency chains. The theoretical framework of BBM rheology and stress integration algorithms followed the formulation introduced by Ólason et al. [7]. The audit covered key numerical stages: Lagrangian mesh initialization, strain rate tensor computation, BBM damage criteria evaluation, shear stress integration leading to LKF emergence, and diagnostic variable export.

4.6 Track 2: Direct copernicus inversion via brittle BBM rheology

An inversion pathway deriving internal stress states directly from external forcings and ice parameters was investigated, mirroring the formulation in Ólason et al. [7]. This approach couples ice thickness and drift velocities through BBM rheology equations.

The total strain rate ϵ decomposes into elastic and viscous components ($\epsilon = \epsilon_E + \epsilon_v$). When the mean normal stress $\sigma_N = \frac{1}{2}(\sigma_{11} + \sigma_{22})$ exceeds the compressive strength threshold ($P_{\max}$), a viscous correction factor $\tilde{P}$ triggers plastic yield:

$$\sigma_v = (1 + \tilde{P})\sigma \quad \text{where} \quad \tilde{P} = \begin{cases} P_{\max}/\sigma_N & \text{Regime I: Ridging } (\sigma_N < -P_{\max}) \\ -1 & \text{Regime II: Pure Elastic } (-P_{\max} < \sigma_N < 0) \\ 0 & \text{Regime III: Free Divergence } (\sigma_N > 0) \end{cases} \quad (9)$$

The maximum compressive strength $P_{\max}$ scales non-linearly with ice concentration A and thickness h :

$$P_{\max} = P^* \left(\frac{h}{h_0} \right)^{3/2} e^{-C(1-A)} \quad (10)$$

As this formulation represents a forward sea-ice integration scheme distinct from the characteristic inversion framework requested by the supervisor, this track was omitted.

4.7 Track 3: Shear integration via geometric curvature vs. characteristic formulations

Two competing stress integration formulations were evaluated on an isolated LKF (extracted from RGPS data following spatial interpolation). The local orientation angle β relative to the x-axis and angular deflection $\Delta\beta$ were extracted. Daily-averaged Copernicus wind stress was incorporated as the primary external forcing vector ($a = c(u_{wind}^2 + v_{wind}^2)$).

Significant divergence was observed between the two formulations:

Method 1 (Geometric Curvature formulation): Relies on explicit geometric curvature $\kappa = d\beta/ds$ along curvilinear arc-length s . Incorporating wind force magnitude (f_{wind}) and angle (α_{wind}), shear stress Z is isolated via:

$$Z = \frac{f_{wind} \cdot \sin(\beta - \alpha_{wind} \pm \lambda)}{4\kappa} \quad \text{with} \quad \lambda = \frac{\pi}{4} + \frac{\phi}{2} \quad (11)$$

Method 2 (logarithmic characteristic formulation): Method 1 suffers from numerical singular behavior along linear fracture segments ($\kappa \rightarrow 0$). Applying Mohr-Coulomb plasticity theory via the Method of Characteristics, the relative shear stress variation (Z) as a function of angular deflection ($\Delta\beta$) yields:

$$\frac{Z + \Delta Z}{Z} = \exp(\pm 2 \tan \phi \cdot \Delta\beta) \quad \text{with internal friction angle } \phi = 15^\circ \quad (12)$$

Since I applied both methods to the LKF geometry, I didn't obtain coherent results to present. I got very different results from the two approaches.

5 Final Track - Reconstruction of shear stress profiles: a joint characteristic and LKF geometry approach

In this section, we couple two equations from Erlingsson (1988) into a Python script to map the two-dimensional shear stress field. Under the Mohr-Coulomb criterion, the system becomes hyperbolic and yields two families of characteristics. The first family (s_1) follows the active LKF, while the second (s_2) corresponds to the conjugate characteristic direction.

$$dZ + 2 \tan(\phi) Z d\beta = \sin(\phi) a_1 ds_1 \quad (1)$$

$$dZ - 2 \tan(\phi) Z d\beta = \sin(\phi) a_2 ds_2 \quad (2)$$

Subject to the spatial propagation constraint:

$$\frac{ds_{1,2}}{dy} = \tan(\beta \pm 2\lambda) \quad (3)$$

where ϕ is the internal friction angle, β is the principal stress orientation, $a_{1,2}$ represent external forcing vectors (wind/current drag), and $\lambda = \pi/4 + \phi/2$ is the deflection angle (angle between the two characteristics).

5.0.1 Initial Interpretation: Branching Tree Reconstruction Algorithm

My initial interpretation was that stress evolution branches into a decision tree rather than following a single trajectory, as each fracture bifurcation offers a binary choice between conjugate directions ($\pm\lambda$). I initially designed the algorithm as follows:

1. At each discrete node along the fracture, both candidate principal stress orientations β leading to plastic failure are computed.
2. A branching decision tree is deployed step-by-step using an adaptive curvilinear step size ($ds = 1$) to explore all prospective spatial trajectories.
3. By constraining the pathing with observed satellite fracture geometry (local angles extracted from RGPS), the algorithm dynamically selects the physical branch within the tree.
4. This selection reconstructs the exact historical shear stress profile Z along the LKF track.

Starting from an observed fracture line with known geometry and local shear stress, and applying external forcing fields (such as wind stress), this method was intended to solve the inverse mechanical problem: reconstructing the internal shear stress Z across the entire crack network. By integrating these local stress profiles step-by-step along the characteristic directions. However, this initial conceptualization proved to be inaccurate.

5.1 Mathematical formulation and algorithm for shear integration along characteristic line

In this new setup, we model the trajectory deviation of a characteristic line (s_2) originating from an initial straight baseline (s_1 , aligned along the X-axis) subjected to a constant, steady wind vector blowing vertically along the Y-axis. To isolate the primary mechanical response, the wind is assumed to be the dominant forcing vector, allowing other external loads to be neglected. The deflection angle λ is defined by:

$$\lambda = \frac{\pi}{4} + \frac{\phi}{2} \quad (4)$$

where ϕ is the internal friction angle of the sea ice matrix.

The iterative numerical loop integrates along s_2 using a spatial step size $ds_2 = 0.01$ m, starting with an initial bifurcation angle $\theta_{P1} = 2\lambda$. At each discrete node P_1 , the local component of the external wind forcing a_2 acting normal to the characteristic path is evaluated as:

$$a_2 = a_{\text{wind}} \cdot \cos\left(\theta_{P1} - \frac{\pi}{2}\right) \quad (5)$$

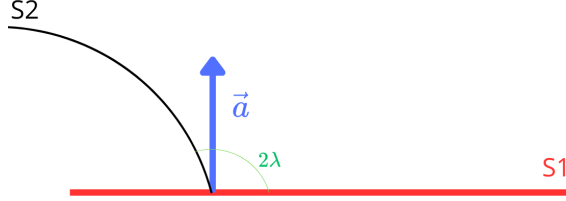


Figure 1: Characteristics Scheme

This external forcing directly drives the differential variation of maximum shear stress dZ through the hyperbolic differential relation:

$$dZ = 2 \tan(\phi) Z d\theta + \sin(\phi) a_2 ds_2 \implies Z_{P2} = Z_{P1} + dZ \quad (6)$$

yielding the shear stress state at the subsequent node P_2 .

Using the updated shear stress Z_{P2} , the local geometric curvature κ of the characteristic trajectory is computed as:

$$\kappa = \frac{a_2}{4Z_{P2}} \cos(\phi) \quad (7)$$

This curvature determines the angular orientation shift $\theta_{P2} = \theta_{P1} + \kappa \cdot ds_2$ at node P_2 , whose spatial coordinates are updated via forward Euler integration:

$$x_{P2} = x_{P1} + ds_2 \cos(\theta_{P2}) \quad (8)$$

$$y_{P2} = y_{P1} + ds_2 \sin(\theta_{P2}) \quad (9)$$

The algorithm then advances by replacing P_1 with P_2 for the next iteration.

Under continuous wind action along the positive Y-axis, the projection term a_2 remains positive throughout trajectory progression. Consequently, $dZ > 0$, yielding a strictly non-decreasing profile for the shear stress Z . As the characteristic line bends under local curvature κ , its path gradually become orthogonal with the vertical wind vector ($\theta \rightarrow \pi$), as illustrated in Figure 2. In this asymptotic orientation, the normal wind component a_2 vanishes, driving the differential source term to zero ($dZ \rightarrow 0$). The accumulated shear stress Z then plateaus at a constant asymptotic value, representing the maximum steady-state mechanical loading along the path.

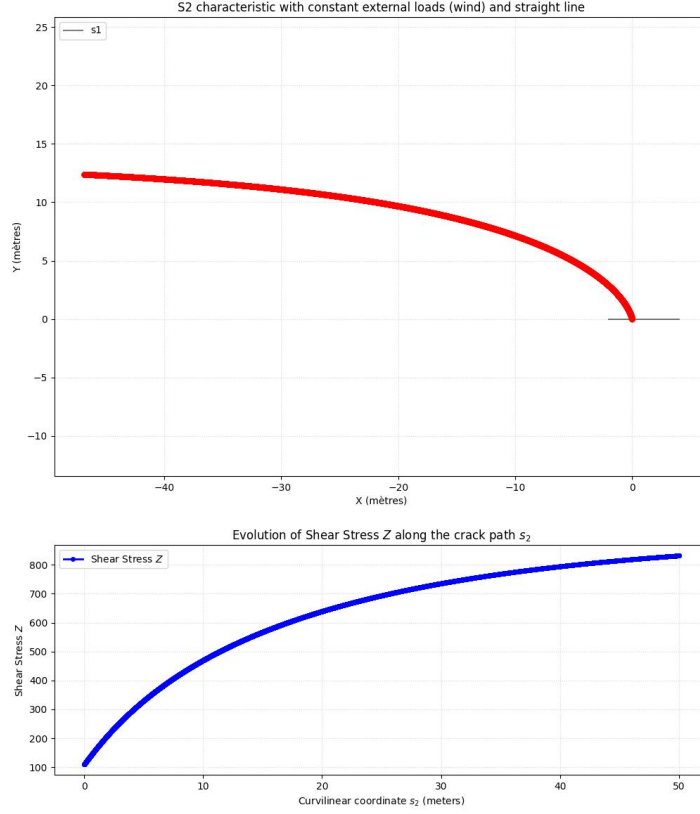


Figure 2: Geometric trajectory and stress progression of the s_2 characteristic using Method 1 ($ds_2 = 0.01$ m).

Method 2: Isolated initial bifurcation (2λ forced order inversion)

Upon auditing the original iterative loop (Method 1), a minor numerical bug was identified: the initial plotting angle deviated slightly from the theoretical value of 2λ because the angular orientation was prematurely recalculated during the very first integration step before establishing the initial baseline geometry.

To resolve this issue and guarantee exact physical consistency at the origin, the loop execution order was inverted by strictly isolating the initial geometric bifurcation from the curvature-driven iteration:

- **Step 0 (Pre-loop initialization):** The initial segment $P_0 \rightarrow P_1$ is evaluated *prior to* entering the main loop. The orientation of this first segment is strictly fixed to the theoretical bifurcation angle $\theta_0 = 2\lambda$. The initial shear stress Z_1 is subsequently derived directly from the local wind component acting along this forced segment.(

$$dZ = 2 \tan(\phi) Z d\theta + \sin(\phi) a_2 ds_2 \quad \implies \quad Z_{P1} = Z_{P0} + dZ \quad (10)$$

)

- **Step 1 and subsequent iterations (Inside the Loop):** Proceeding from node P_1 , the algorithm executes the reordered sequence at each iteration step.

1. **Draw and save node P_1 , coordinate and shear** We save coordinate, shear calcul of P_1 to draw it a the end of the program.
2. **Curvature calculation:** Compute local curvature κ using the shear stress state Z from the previous node.
3. **Angle update:** Update propagation angle: $\theta_{\text{new}} = \theta + \kappa \cdot ds_2$.
4. **Spatial node advancement:** Compute spatial coordinates of the next node P_2 .
5. **Shear stress update:** Integrate and update the shear stress Z corresponding to the new orientation.

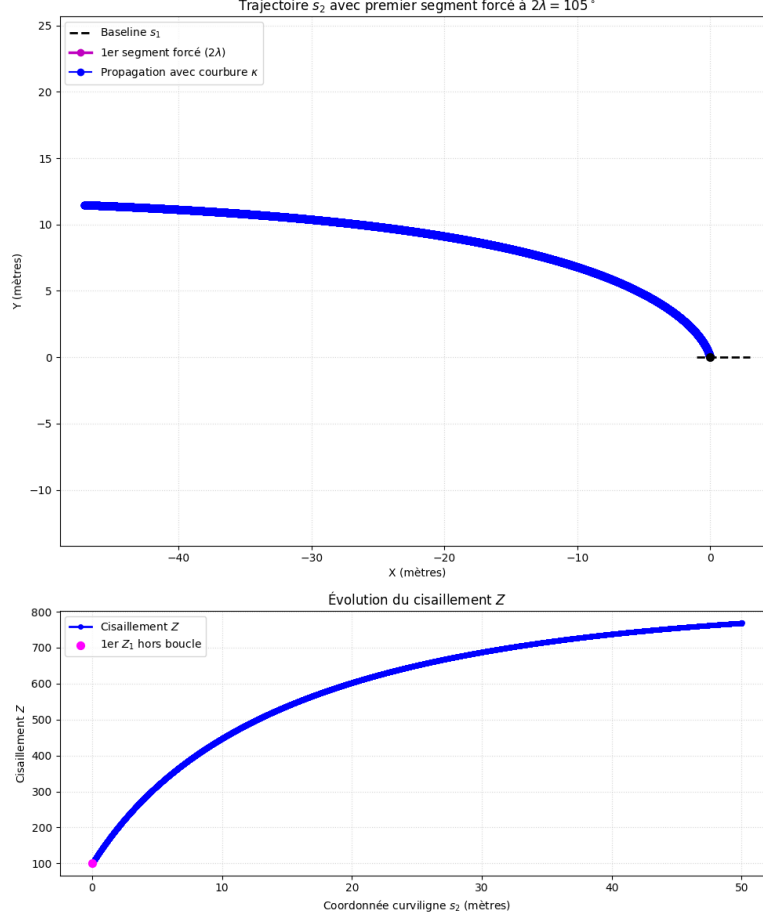


Figure 3: Geometric trajectory and stress progression of the s_2 characteristic using Method 2.

Comparing Figure 2 (Method 1) and Figure 3 (Method 2) highlights key structural and physical behaviors:

- **Initial Segment Behavior:** Method 2 exhibits a sharp initial step-like jump in Z due to the strictly forced 2λ angle on segment $P_0 \rightarrow P_1$, whereas Method 1 displays a smoother initial transition.
- **Angle-Curvature Lag Effect:** In Method 2, evaluating θ_{new} using the curvature $\kappa(Z_{\text{previous}})$ from the *preceding* node introduces a 1-step numerical phase lag between stress evaluation and trajectory deflection.

Visually, both formulations align towards the same mechanical steady state, confirming that the initial 2λ order inversion in Method 2 does not alter the overall path geometry or stress saturation behavior. A quantitative evaluation of numerical stability and step-size dependency ($ds_2 \rightarrow 0$) is detailed in the following convergence section.

5.2 Algorithmic optimization: transition to dynamic convergence criteria

In the initial implementation, spatial integration was executed using a fixed-length `for` loop over a predefined number of steps. However, preliminary testing revealed that this static approach was inadequate: using an insufficient number of iterations paired with a relatively coarse step size (ds_2) prevented the integration path from fully reaching its asymptotic steady state.

To ensure that the shear stress Z reaches its physical saturation plateau without relying on arbitrary number of iteration, the static loop was replaced with a dynamic `while` loop governed by an explicit convergence threshold on the differential stress increment:

$$|dZ| < 10^{-5} \text{ N/m} \quad (11)$$

Under this dynamic criteria, the algorithm continuously steps along the characteristic path until the normal forcing component a_2 vanishes ($a_2 \rightarrow 0$) as the trajectory aligns with the wind direction, causing dZ to drop below the 10^{-5} numerical threshold.

5.3 Mesh convergence analysis

A spatial convergence study is essential to ensure that the integrated shear stress profiles and trajectory geometries reflect physical reality rather than numerical artifacts. If the spatial step size ds_2 is chosen too coarse, cumulative truncation errors lead to physically inaccurate stress predictions and distorted characteristic paths.

To evaluate solver stability, we analyze the mechanical state (Z) and the vertical position (Y) evaluated at a fixed curvilinear distance $L = 5 \text{ m}$ along the s_2 line. The spatial step size ds_2 is progressively refined across four orders of magnitude, ranging from $ds_2 = 1.0 \text{ m}$ down to $ds_2 = 10^{-4} \text{ m}$. Each plot presents dual vertical axes: the accumulated shear stress $Z(L = 5 \text{ m})$ on the left axis and the corresponding vertical coordinate $Y(L = 5 \text{ m})$ on the right axis.

Under zero initial stress ($Z_0 = 0$) When initializing the integration without background stress ($Z_0 = 0$), both Method 1 (Figure 4) and Method 2 (Figure 5) exhibit a collapse: as $ds_2 \rightarrow 10^{-4} \text{ m}$, both the shear stress Z and the vertical position Y decay toward zero at $L = 5 \text{ m}$. This behavior is physically coherent. Next, we use a non-zero initial shear stress.

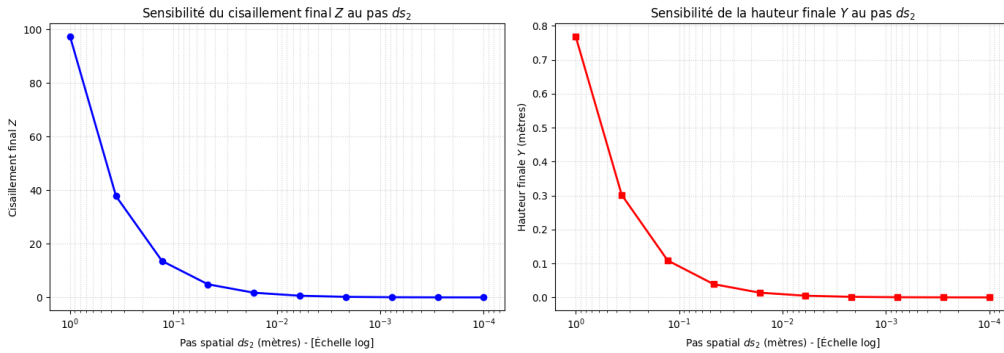


Figure 4: Method 1 convergence study at fixed distance $L = 5 \text{ m}$ initialized with $Z_0 = 0$. Both shear stress Z (left axis) and vertical position Y (right axis) collapse toward zero as $ds_2 \rightarrow 0$.

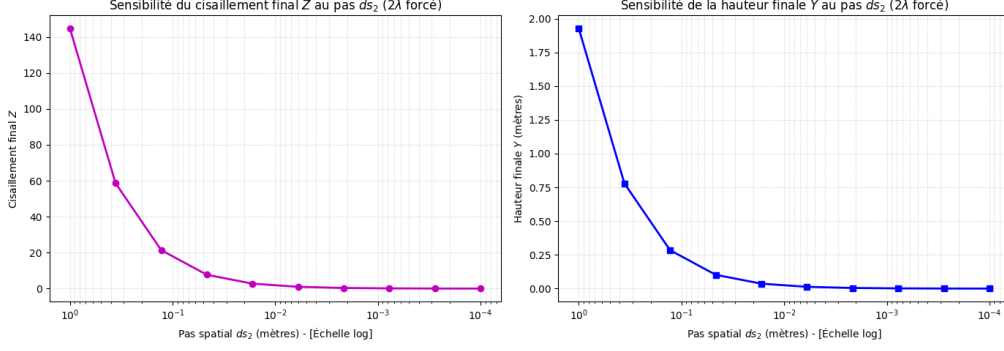


Figure 5: Method 2 convergence study at fixed distance $L = 5$ m initialized with $Z_0 = 0$, exhibiting the same asymptotic decay toward zero as $ds_2 \rightarrow 0$.

Physical asymptotic stability under initial stress ($Z_0 = 110$ N/m) When imposing a realistic baseline shear stress ($Z_0 = 110$ N/m), the solver stabilizes upon refinement. For both Method 1 (Figure 6) and Method 2 (Figure 7), as $ds_2 \rightarrow 10^{-4}$ m, the shear stress evaluated at distance $L = 5$ m converges toward a stable physical value of $Z \approx 330$ N/m (with $Y \approx 3.2$ m). Note that $Z \approx 330$ N/m is not the final steady-state plateau of the infinite line, but the exact stress reached at the fixed arc-length marker $L = 5$ m.

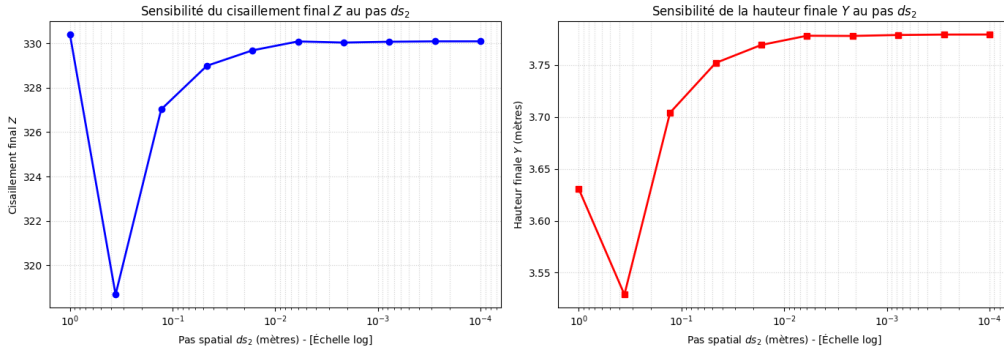


Figure 6: Method 1 convergence study at fixed distance $L = 5$ m initialized with $Z_0 = 110$ N/m. As $ds_2 \rightarrow 0$, Z smoothly converges to 330 N/m and Y to 3.2 m.

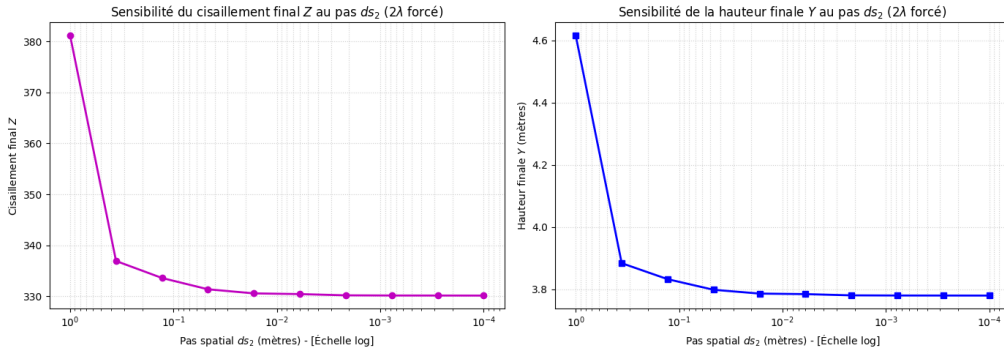


Figure 7: Method 2 convergence study at fixed distance $L = 5$ m initialized with $Z_0 = 110$ N/m, confirming identical asymptotic convergence ($Z \approx 330$ N/m) to Method 1.

The perfect agreement between Method 1 and Method 2 under $Z_0 = 110$ N/m confirms that both algorithmic formulations converge to the exact same physical solution.

5.3.1 Quantitative convergence analysis: log-log error scaling

To evaluate the numerical order of accuracy of both algorithms, a grid refinement study was conducted using 40 logarithmically spaced spatial step sizes $ds_2 \in [5 \times 10^{-5}, 5.0]$ m. We stay with a $L = 5$, so we reduce the mesh by increasing the number of iterations : $N = \frac{L}{ds_2}$. The absolute error E is evaluated relative to the high-precision asymptotic reference value $Z_{\text{ref}} = 330.0957$:

$$E(N) = |Z_{\text{final}}(N) - Z_{\text{ref}}| \quad (12)$$

Method 1: Direct synchronous recalculation

For Method 1, performing a linear regression on the log-log error curve $\log_{10}(E)$ versus $\log_{10}(N)$ yields a measured convergence slope of:

$$p_1 = -1.127075 \quad (13)$$

This value ($p_1 \approx -1.0$) confirms the expected first-order asymptotic accuracy corresponding to the explicit Euler integration scheme.

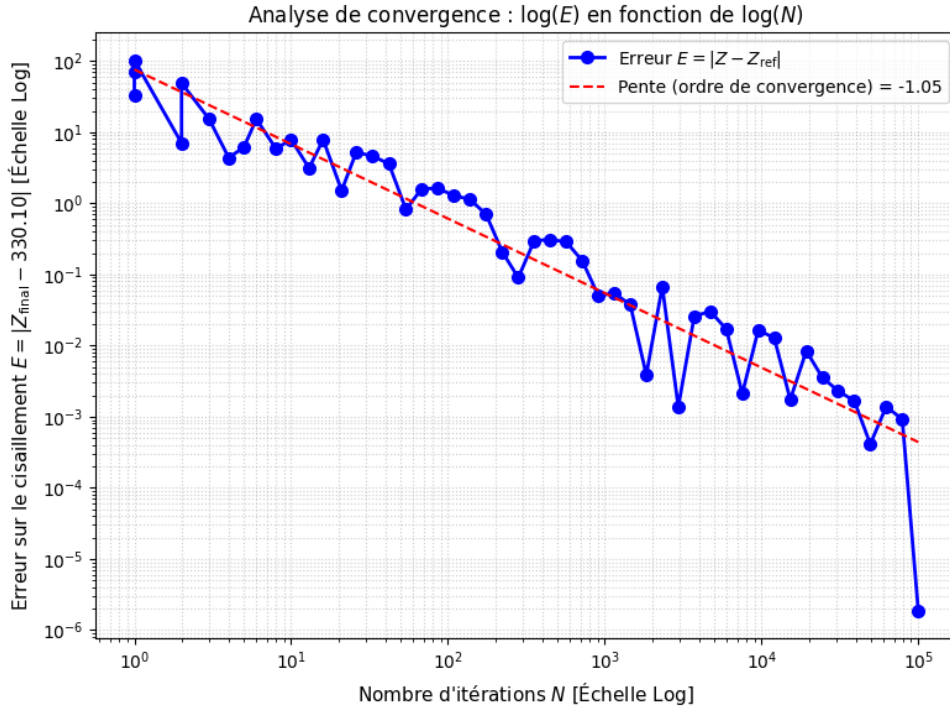


Figure 8: Log-log error plot $\log_{10}(E)$ vs. $\log_{10}(N)$ for Method 1 using $Z_{\text{ref}} = 330.0957$. The empirical order of accuracy is $p_1 = -1.127075$.

Method 2: Isolated initial bifurcation (2λ forced)

Applying the same grid refinement protocol to Method 2 yields a measured convergence slope of:

$$p_2 = -0.955699 \quad (14)$$

This slope is remarkably close to the theoretical first-order line ($p = -1.0$). It demonstrates that neither the localized 1-step phase lag in curvature calculation nor the initial step-like stress jump at 2λ degrades the global spatial convergence rate of the algorithm.

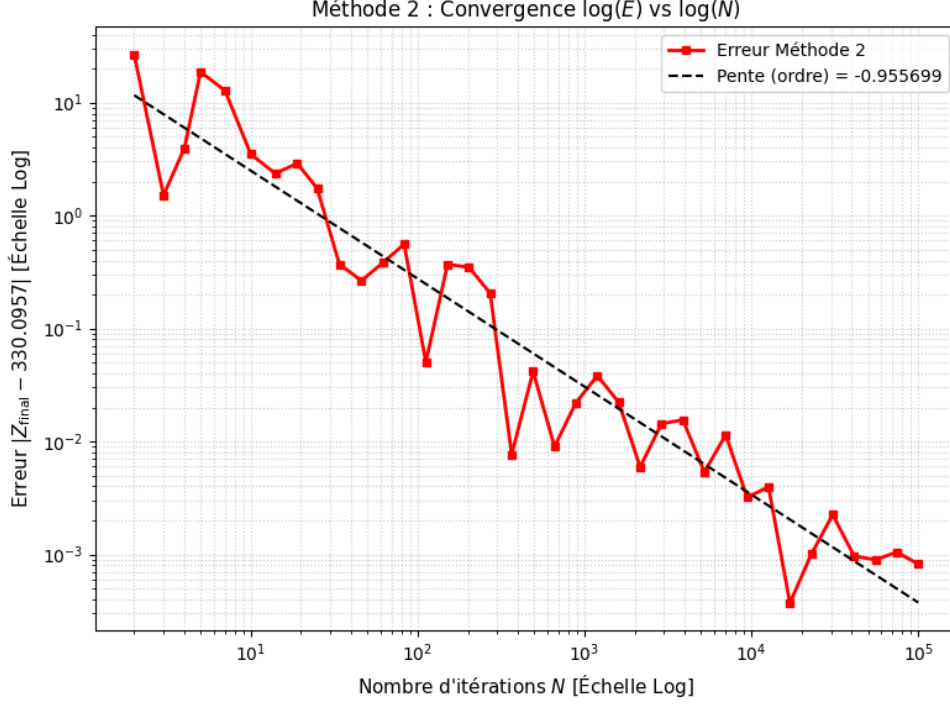


Figure 9: Log-log error plot $\log_{10}(E)$ vs. $\log_{10}(N)$ for Method 2 using $Z_{\text{ref}} = 330.0957$. The empirical order of accuracy is $p_2 = -0.955699$.

5.3.2 Comparative summary and validation

Table 1: Quantitative comparison of numerical convergence parameters for Method 1 and Method 2.

Method	Asymptotic Reference Z_{ref}	Empirical Slope (p)	Asymptotic Scheme Order
Method 1	330.0957	-1.127075	$\mathcal{O}(ds_2^1)$ (First-Order)
Method 2	330.0957	-0.955699	$\mathcal{O}(ds_2^1)$ (First-Order)

The close agreement between both empirical slopes ($p_1 \approx -1.13$ and $p_2 \approx -0.96$) frames the theoretical value $p = -1.0$. This cross-validates both numerical implementations and confirms complete mesh-independence and physical consistency as $ds_2 \rightarrow 0$. This first-order accuracy, while modest compared to higher-order schemes such as Runge-Kutta 4, was considered sufficient here: given the low computational cost per iteration and the fine step sizes achievable (down to $ds_2 = 10$ m), first-order convergence still yields a numerically stable and physically consistent solution, making a more complex scheme unnecessary for the scope of this study.

5.3.3 Final saturation of shear stress Z

Initially, the curve exhibits a value of $Z \approx 330$, which corresponds to a cumulative curvilinear distance of $s_2 = 50.0$ m (reached at iteration $i = 10\,000$ with $\Delta s_2 = 0.005$ m). This value does not represent the final equilibrium state where the shear stress become constant.

It's why we change for a 'while' loop to reach the saturation level. With $i = 125\,412$ (corresponding to $s_2 = 627.06$ m), where the relative variation satisfies $|\Delta Z/Z| < 10^{-6}$, the true final asymptotic shear stress limit for the system is therefore:

$$Z_{\text{final}} \approx 860.45 \quad (15)$$

This confirms that $Z \approx 330$ was merely an intermediate checkpoint, whereas $Z \approx 860.45$ constitutes the actual physical and numerical steady-state threshold along the characteristic s_2 .

5.4 Multi-level saturation of shear stress

To extend the one-dimensional characteristic trajectory s_2 into a full two-dimensional computational grid, a multi-level scanning approach (referred to as *fencing*) is implemented. The spatial discretization and stress transfer follow a structured, step-by-step algorithm:

Baseline initialization (s_1) We always begin with a horizontal straight line LKF with constant shear ($Z=100$) and constant wind value in y axis. The line is divided in increment $ds_1 = 1.0$ m.

Parallel branch propagation (s_2) From each nodal point on s_1 , a characteristic line of the s_2 family is propagated upwards using a fine spatial step $ds_2 = 0.005$ m under a vertical wind loading $a_{\text{wind}} = 150.0$. The local shear stress Z and turning curvature κ are updated until s_2 re-aligns with the vertical wind flow ($\theta \rightarrow \pi$). All s_2 will reach the same saturation level of shear stress. In what follow, we will call this level saturation, a 'fence'. In figure 10 is shown the first fence where all s_2 reached saturation state.

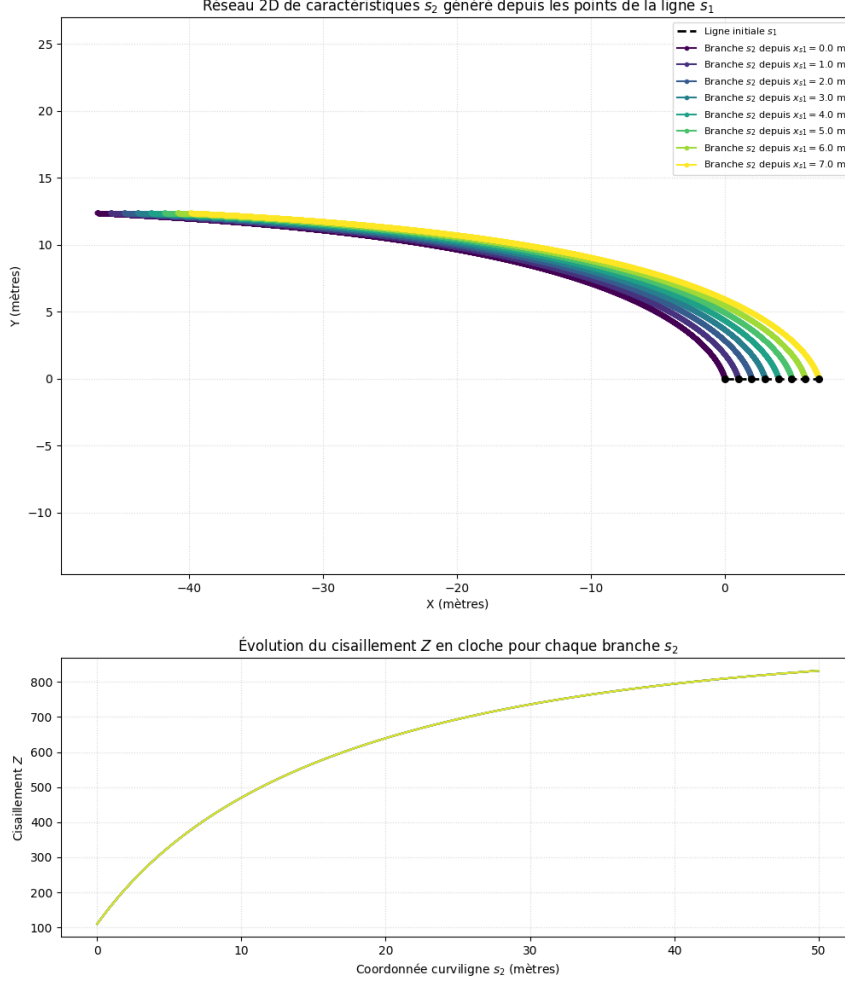


Figure 10: First fence level : drawing of multiple s_2 along the LKF s_1 (with $ds_2 = 0.005$ m, $a_{\text{wind}} = 150.0$; $N_{s1} = 8$ baseline nodes)

Boundary interception (*fence*): After reaching the first fence, we can do the same with taking the final value of shear on the first fence as the new initial value.

$$Z_{\text{init}}^{\text{level } k} = Z_{\text{final}}^{\text{level } k-1} \quad (16)$$

with k : the level of the fence. The horizontal fence represent the new s_1 baseline. This ensure the continuity of the shear stress field $Z(x, y)$ while allowing the characteristic network to adapt dynamically to curvature variations across successive fence.

Figure 11 depicts the multi-level grid extension across six stacked levels ($N_{\text{fences}} = 6$) obtained by recursively repeating the single-level propagation scheme. For each successive fence level k , the initial shear stress assigned to the starting nodes is updated to match the maximum asymptotic value accumulated at the top of the underlying level ($Z_{\text{init}}^k = Z_{\text{max}}^{k-1}$), thereby enforcing mechanical continuity across the entire 2D network.

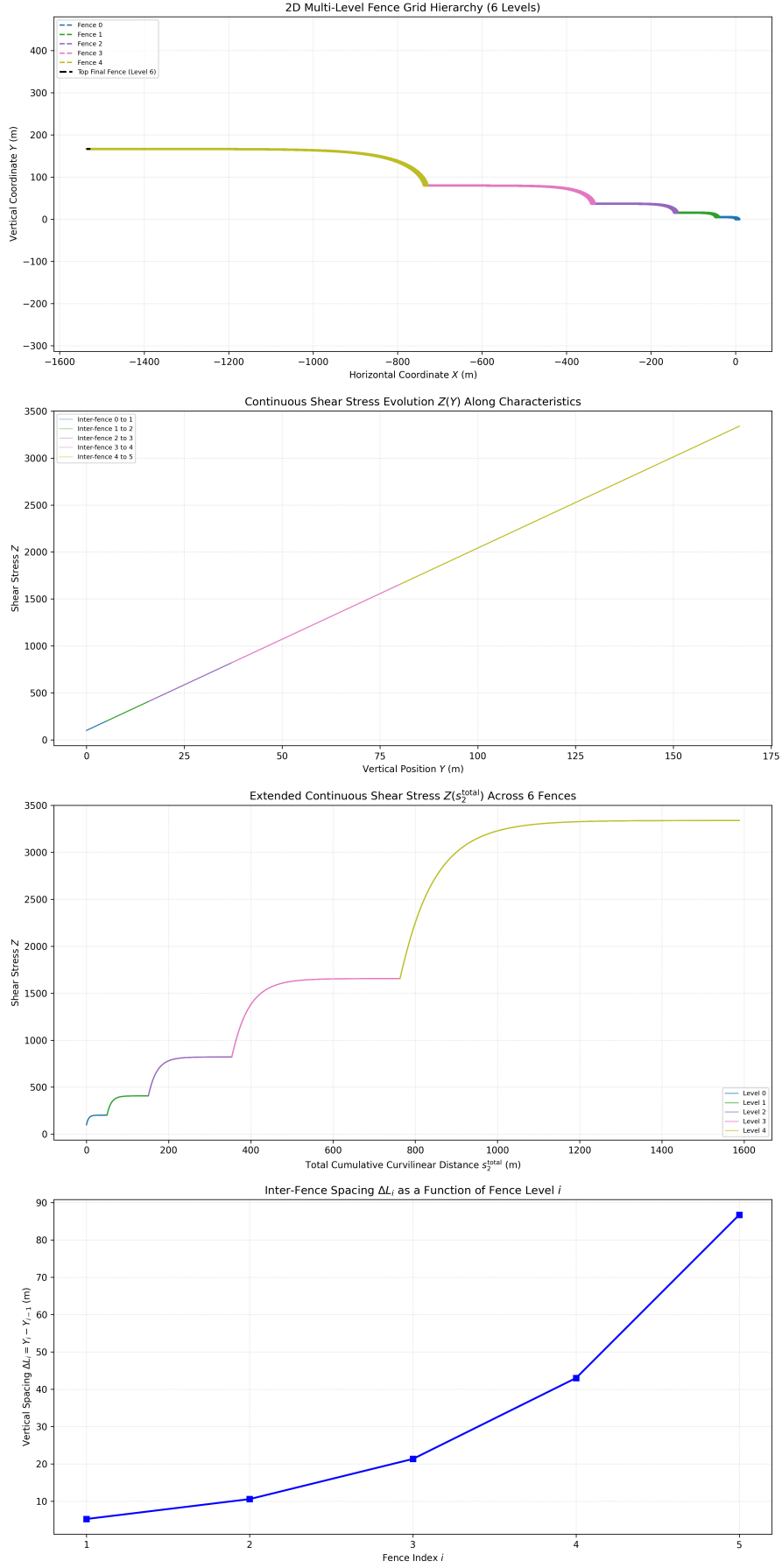


Figure 11: Multi-fence shear integration along s_2 (6 fence levels: (a) 2D spatial grid $Y(X)$, (b) shear stress profile vs. vertical coordinate $Z(Y)$, (c) extended continuous shear stress vs. total curvilinear distance $Z(s_2^{\text{total}})$, and (d) inter-fence vertical spacing ΔL_i as a function of fence level i .

2D Spatial Trajectory Domain (Y vs. X): The upper plot shows the physical trajectory characteristic paths s_2 originating from $N_{s1} = 10$ baseline nodes across six consecutive fence boundaries. Under the vertical wind field a_{wind} , each branch bends progressively from its initial angle $\theta = 2\lambda = 52.5^\circ$ until aligning vertically ($\theta \rightarrow \pi$). As the fence level increases, the paths elongate significantly, broadening the vertical extent of each layer. **Shear Stress Evolution vs. Vertical Position (Z vs. Y):** The second plot illustrates the continuous accumulation of shear stress $Z(Y)$ between inter-fence stages. This is a straight line, so Z might be proportional to axis Y .

Extended continuous shear accumulation (Z vs. s_2^{total}): The third plot tracks the shear stress Z along the entire accumulated curvilinear path s_2^{total} from Level 0 to Level 6. This unified curve demonstrates that stress growth is smooth and non-decreasing across the network. Each fences are truly visible,

Inter-Fence Spacing Scaling (ΔL_i vs. i): The bottom plot displays the vertical inter-fence spacing $\Delta L_i = Y_i - Y_{i-1}$ as a function of the fence index i .

2D Shear map: By doing many curves and many fences, the iterated process can recursively assemble the complete 2D shear grid.

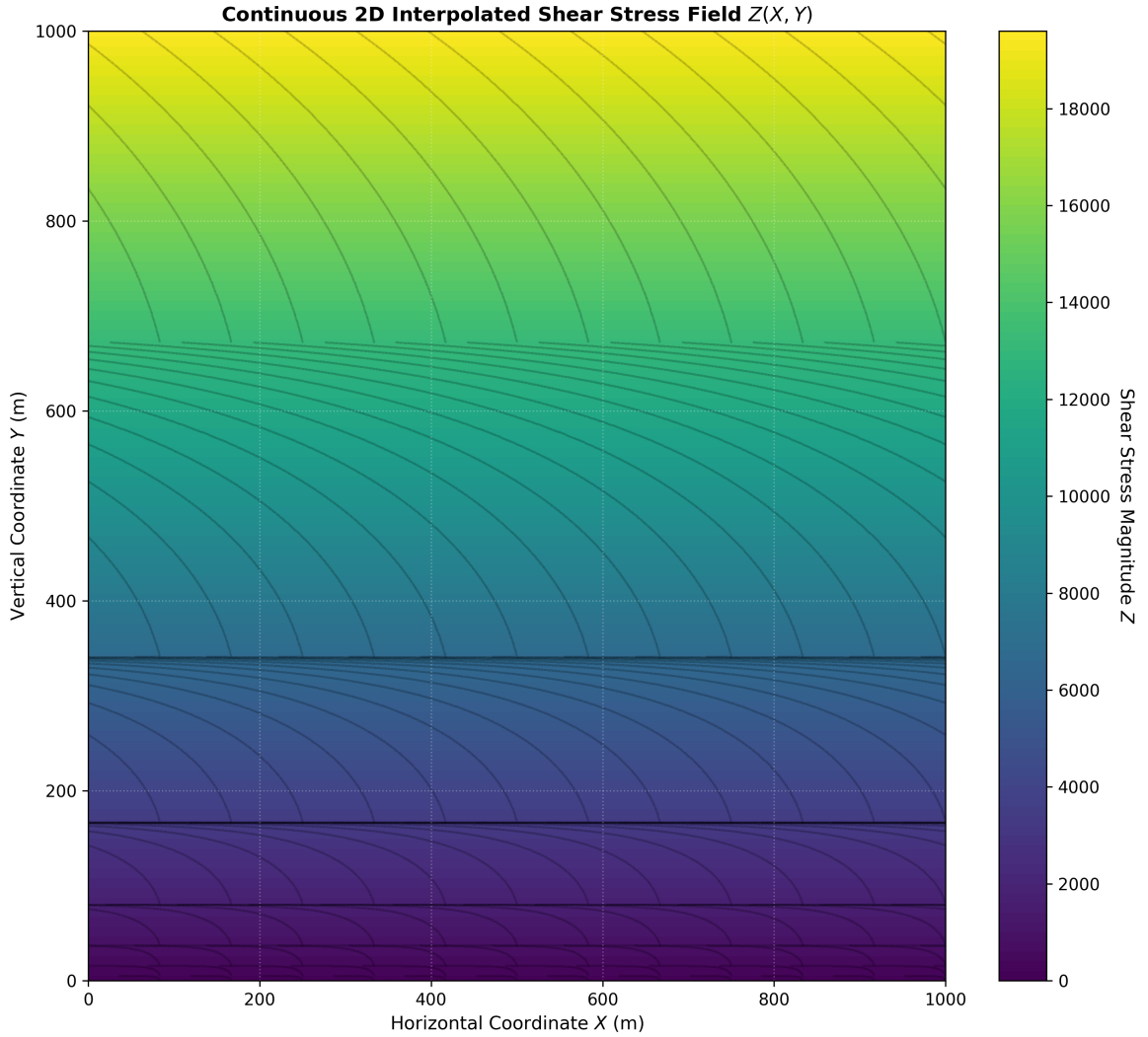


Figure 12: Shear maps of 1km*1km

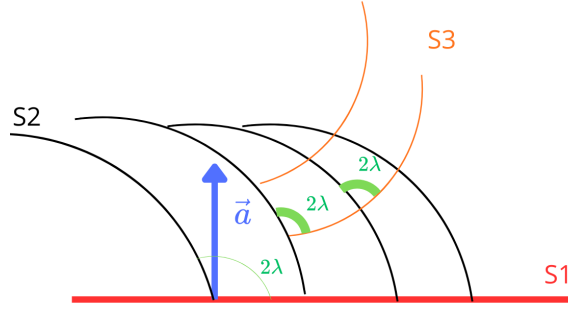


Figure 13: Schema of S3

5.5 Computation of a third characteristic S3

5.5.1 Mathematical framework and propagation logic

To simulate crack propagation across the structural branches (S_2), we employ a characteristic-based method. The geometry of the S_2 branches is discretized and reconstructed as smooth continuous functions using 'UnivariateSpline' interpolation. At any point P along an S_2 branch, the new crack segment direction (θ_{ds3}) is computed by applying an angular offset of -2λ relative to the local tangent angle (θ_{ds2}): \

$$\theta_{ds3} = \theta_{ds2} - 2\lambda \quad (17)$$

The crack path is then projected until it intersects the next S_2 branch. The intersection point is determined numerically by finding the roots between the linear trajectory and the spline curve of the subsequent branch.

5.5.2 Initial validation

Before simulating full propagation chains, the algorithm was validated starting from a single point. Figure 14 illustrates the 2λ angle formed between S_2 and the newly generated S_3 segment, confirming the successful detection of the intersection point (highlighted in red).

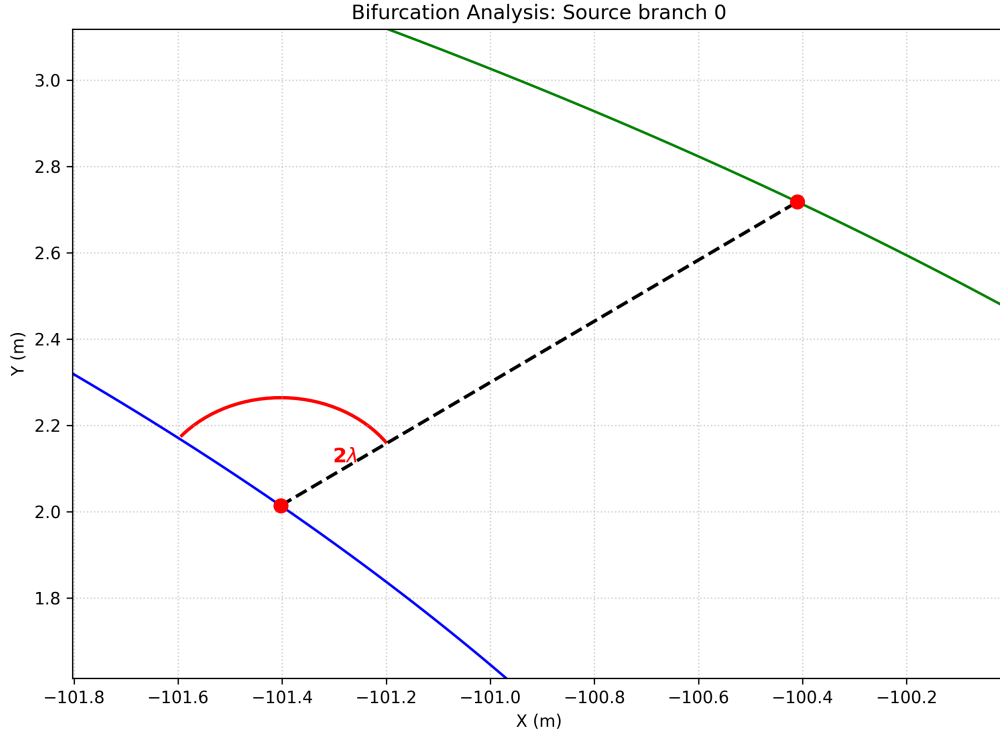


Figure 14: Angle validation.

5.5.3 Fracture chain analysis (S_3)

The methodology was then extended to generate complete multi-layer fracture chains (S_3), tracking the continuity of the rupture path across successive branches.

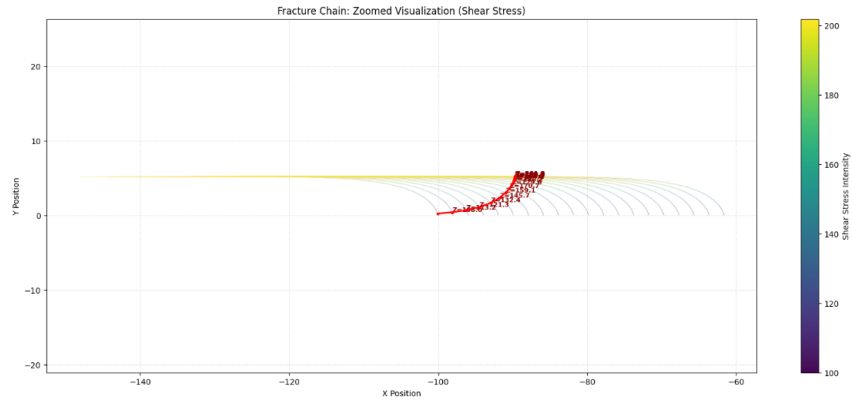


Figure 15: Visualization of a complete S_3 fracture chain. Here is represented the continuous propagation across several branches, highlighting the manner in which the fracture connects successive intersection points.

Figure 15 shows an isolated S_3 trajectory, demonstrating continuous propagation and directional changes driven by the branch geometry. To assess the network's structural impact, propagation was compared across two fences. This comparison allows me to test the robustness of the propagation as the branch network expands.

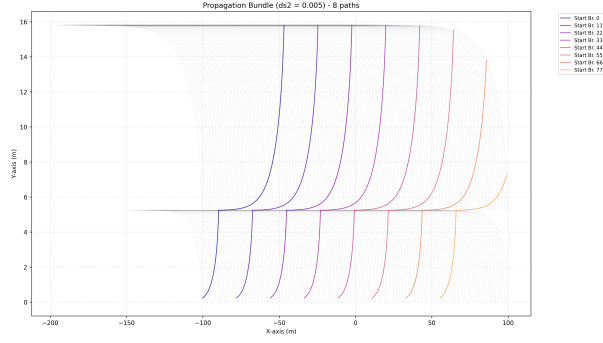


Figure 17: Bundle of 8 propagation paths

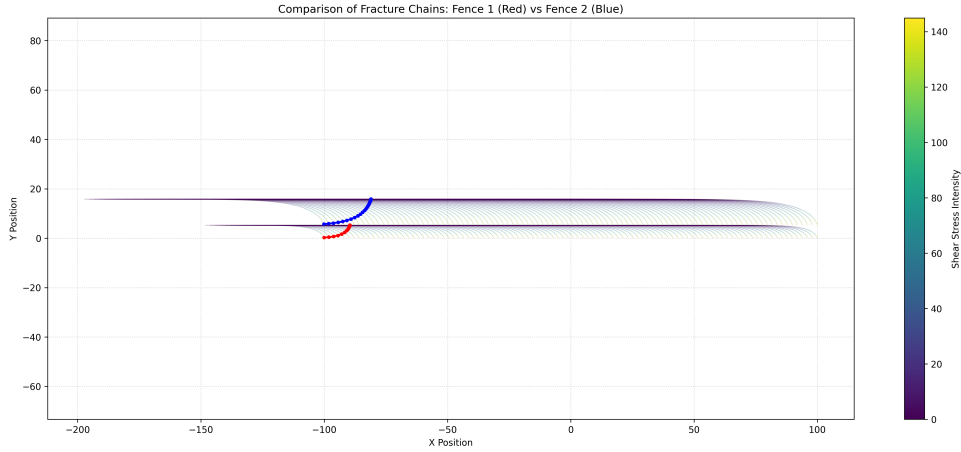


Figure 16: Comparison of S_3 fracture chains across two fences. The red chain represents the primary propagation, while the blue (dashed) chain represents the response on the second fence. We used this configuration to highlight differences in structural behavior depending on the initial positioning.

Figure16 shows a clear divergence between the red and blue paths, confirming that local branch geometry strongly dictates the propagation direction across multi-layer networks.

Finally, Figure17 exhibits a similar pattern. Although different axis scaling makes the curves appear visually distinct, a clear transition occurs: initially horizontal paths reorient and align vertically as they cross successive fences. It's what we are searching for.

5.6 Relation between Delta Z/Z vs phi

To characterize the influence of the injection angle ϕ on the crack propagation, we analyze the final ratio defined by $\Delta Z/Z_{final}$. The simulation results indicate that this ratio does not scale linearly with the angle ϕ ; rather, it exhibits a concave saturation trend, approaching a physical limit as the angle increases.

To capture this behavior, we employ a non-linear saturation model defined by the following exponential function:

$$R(\phi) = A - B \cdot \exp(k \cdot \phi) \quad (18)$$

where A represents the asymptotic limit of the propagation ratio, and k governs the rate of saturation.

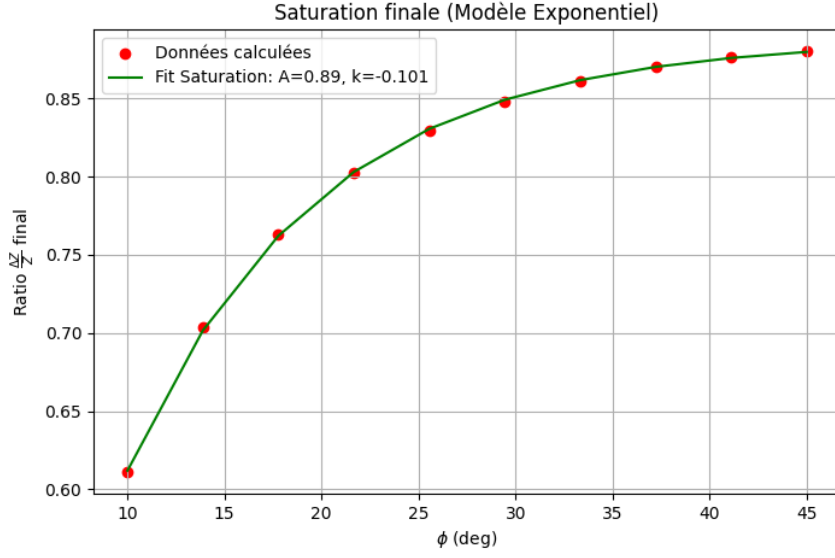


Figure 18: Final propagation ratio $\Delta Z/Z$ as a function of ϕ . The green curve represents the exponential saturation fit, yielding an asymptotic limit of $A \approx 0.89$.

The fitting results, illustrated in Fig. 18, demonstrate a high degree of correlation between the simulation data and the proposed exponential model. The parameter $A \approx 0.89$ suggests the existence of a geometric constraint within the system that prevents the propagation ratio from exceeding this value, regardless of further increases in ϕ .

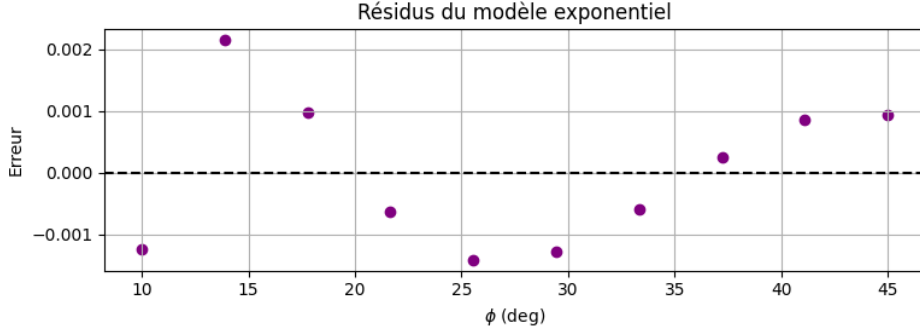


Figure 19: Residual analysis of the exponential saturation model. The distribution of errors confirms the accuracy of the fit, with residuals consistently bounded within the range of ± 0.002 .

The validity of this model is further confirmed by the residual analysis presented in Fig. 19. The distribution of residuals is centered around zero, with errors bounded within a narrow range ($\approx \pm 0.002$). This minimal deviation indicates that the exponential saturation model effectively captures the underlying non-linear physics of the crack propagation process without systematic bias.

5.7 Relation between Delta Li vs phi

A relationship between Delta Li and phi is trying to be found. However, it's more complicated that it seems. Delta i depends on i (the number of the fence) and ϕ (friction angle). The figure 20 represents :

- ΔL_i vs i for each ϕ

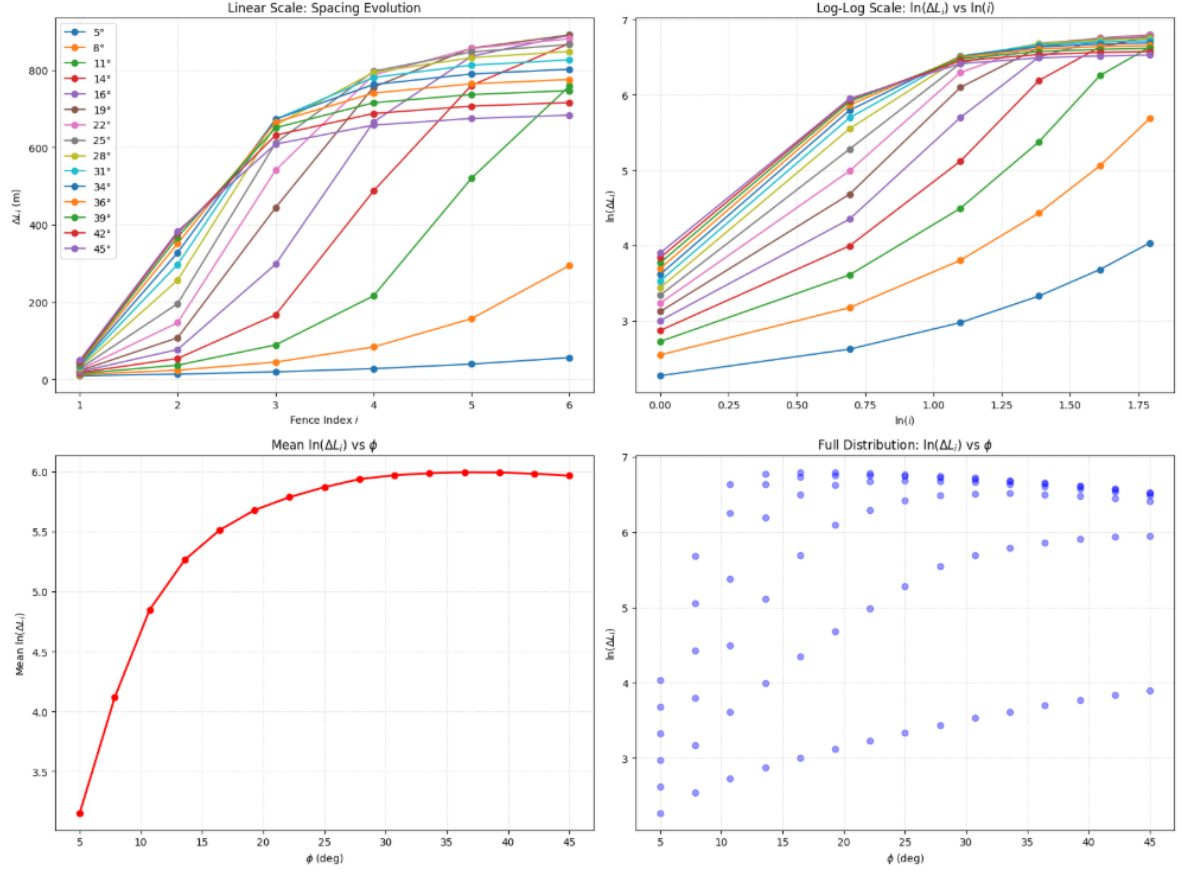


Figure 20: Delta Li vs phi; (a) Evolution of Delta i vs i for each phi; (b) log(delta Li) vs log(i); (c) mean ln(delta li) vs phi; (d) full distribution of ln(delta li) vs phi

- the same with logarithm
- the mean $\ln(\Delta L_i)$ vs ϕ
- full distribution of $\ln(\Delta L_i)$ vs ϕ

An evident relation cannot be found with these results.

5.8 LKF propagation simulation and wind stress integration

To investigate the mechanical behavior of LKFs, we implemented a numerical simulation based on the structural properties of a specific LKF identified in the RGPS (Radarsat Geophysical Processor System) dataset. We selected a specific LKF ID, as its location on the edge of the domain provided a distinct boundary condition for the study.

The simulation utilizes a 3-day window of RGPS data, covering the period from April 2nd to April 4th, 2007. Atmospheric forcing was incorporated by calculating the mean wind stress field over this interval. The wind components were interpolated from the atmospheric dataset onto the LKF geometry using a linear interpolation method to ensure spatial consistency.

We initiate our programm that compute s_2 on multiple fences with :

- The initial shear along the LKF acquired from RGPS data.
- The initial orientation angle θ_0 computed along the LKF track.
- . The characteristic step size ds_2 along the trajectory.

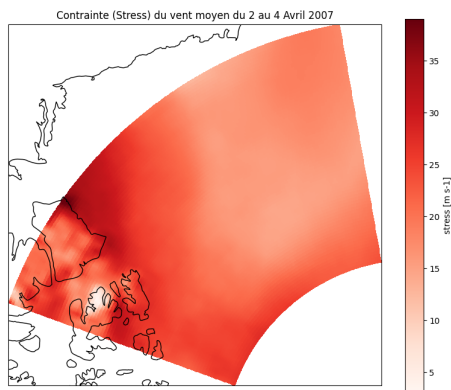
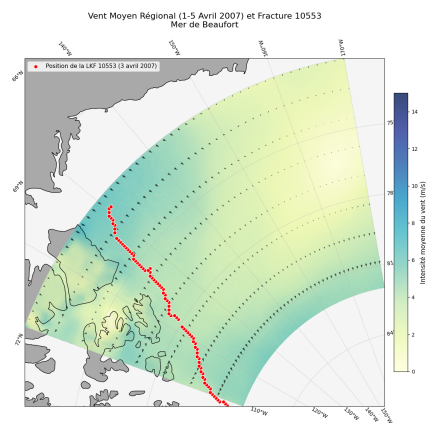
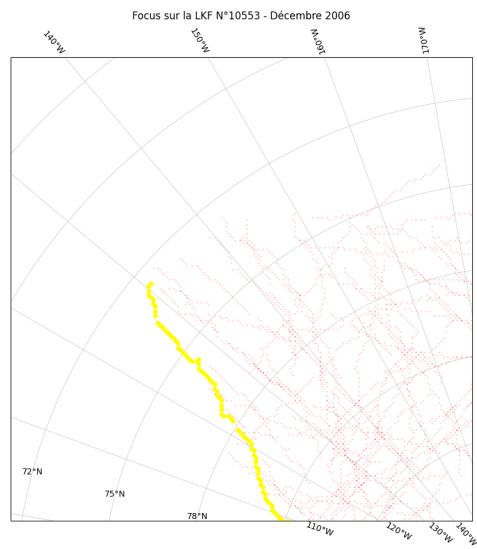


Figure 21: Shear stress from Copernicus

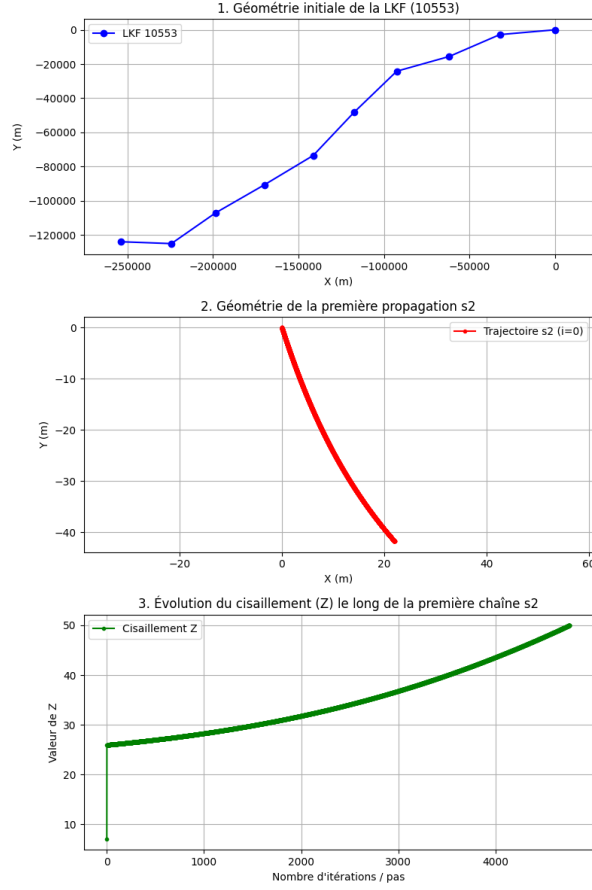


Figure 22: LKF geometry , first s2 geometry, Shear along s2

- The normal atmospheric forcing component a_2 computed from Copernicus surface wind vectors.

Despite the development of this framework for integrating RGPS-derived geometry with atmospheric stress, the simulation did not yield conclusive final results. The model does not converge towards one value and the shear still increase as we see in figure 22 . The model stay in a developmental stage. Future work will focus on refining the stability of the iteration loop and performing a sensitivity analysis on the coupling between wind stress and LKF deformation.

Perpectives and limitations

Several limitations should be acknowledged. First, the final coupled simulation (Section 5.8) combining real RGPS geometry with Copernicus wind forcing did not reach a stable equilibrium: the shear stress Z kept increasing without saturating, unlike the idealized cases of Section 5.3–5.6. This discrepancy likely stems from the simplifying assumption of a constant, unidirectional wind field, which does not reflect the spatial and temporal variability of real atmospheric forcing over the 3-day averaging window. A more realistic treatment would require using the instantaneous (rather than time-averaged) wind field, or reducing the averaging window to better capture short-term forcing variability.

Second, the method was never validated against independent, ground-truthed shear stress measurements. The internal consistency of Method 1 and Method 2 (Section 5.3) demonstrates numerical robustness, but not physical accuracy: both methods could converge to the same self-

consistent — yet physically incorrect — solution if a systematic bias exists in the underlying assumptions (e.g., the choice of internal friction angle $= 15^\circ$, or the neglect of ocean drag and ice-ice interaction forces). Comparing reconstructed Z profiles against BBM-rheology outputs from neXtSIM (Track 1) or against independently derived stress proxies would be a natural next step to assess accuracy rather than just convergence.

Third, the relationships explored between Z/Z , Li and the friction angle (Sections 5.6–5.7) remain only partially conclusive: while an exponential saturation law was fitted for Z/Z , no clear analytical relationship could be established for Li , suggesting that additional physical variables (e.g., local curvature history, wind angle relative to the fracture) may need to be incorporated into the model.

Future work should therefore prioritize: (i) validating the method against real deformation/stress observations, (ii) testing sensitivity to time-varying (non-constant) wind forcing, and (iii) extending the friction-angle analysis with a broader parameter sweep to identify the missing physical drivers of Li .

6 Conclusion

This work allowed me to gain a deeper understanding of Erlingsson’s theory regarding Linear Kinematic Features (LKFs). Using satellite data, I developed an algorithm capable of integrating shear maps under simplified conditions. Given more time, I could have further refined this method using field observations, which would serve to validate the model by assessing its agreement with real-world shear data. Additionally, the model should be verified by identifying relationships between the key quantities involved.

Overall, this internship enabled me to develop strong professional skills, including working independently, collaborating effectively within a team, and managing complex analytical projects.

References

- [12] CMEMS / IFREMER / CERSAT. *Global Ocean Mean Sea Surface Wind and Stress from Scatterometer and Model*. Copernicus Marine Service (CMEMS). 2020. URL: https://marine.copernicus.eu/p/WIND_GLO_WIND_L4_MONTHLY_012_003.
- [11] CMEMS / MET Norway. *Arctic Ocean Physics Analysis and Forecast (Product ID: ARCTIC_ANALYSISFORECAST_PHY_002_001)*. Copernicus Marine Service (CMEMS). 2021. URL: https://marine.copernicus.eu/p/ARCTIC_ANALYSISFORECAST_PHY_002_001.
- [8] Knut-Frode Dagestad et al. “OpenDrift v1.0: a generic framework for trajectory modelling”. In: *Geoscientific Model Development* 11 (2018), pp. 1405–1420.
- [14a] Björn Erlingsson. “Two-dimensional deformation patterns in sea ice”. In: *Journal of Glaciology* 34.118 (1988), pp. 301–308. DOI: [10.3189/s0022143000007233](https://doi.org/10.3189/s0022143000007233).
- [13] Björn Erlingsson. “The propagation of characteristics in sea-ice deformation fields”. In: *Annals of Glaciology* 15 (1991), pp. 73–80. DOI: [10.3189/1991AoG151731](https://doi.org/10.3189/1991AoG151731).
- [14b] Anton Korosov 2 Hugo Boulze 1 * and Julien Brajard 2. “Classification of Sea Ice Types in Sentinel-1 SAR Data Using Convolutional Neural Networks”. In: *remote sensing* (2020).
- [4] Anton Korosov, Yue Ying, and Einar Ólason. “Tuning parameters of a sea ice model using machine learning”. In: *EGUsphere* (2024). Preprint, pp. 1–25. DOI: [10.5194/egusphere-2024-2527](https://doi.org/10.5194/egusphere-2024-2527).
- [1] Anton Korosov et al. “A climate data record of sea ice age using Lagrangian advection of a triangular mesh”. In: *Earth System Science Data* 18 (2026), pp. 721–740. DOI: [10.5194/essd-18-721-2026](https://doi.org/10.5194/essd-18-721-2026).
- [7] Einar Ólason et al. “A New Brittle Rheology and Numerical Framework for Large-Scale Sea-Ice Models”. In: *Journal of Advances in Modeling Earth Systems* 14.8, e2021MS002685 (2022). BBM Model Foundation. DOI: [10.1029/2021MS002685](https://doi.org/10.1029/2021MS002685).
- [6] Pierre Rampal et al. “On the scaling properties of sea ice deformation”. In: *The Cryosphere* 13.9 (2019), pp. 2457–2474. DOI: [10.5194/tc-13-2457-2019](https://doi.org/10.5194/tc-13-2457-2019). URL: <https://tc.copernicus.org/articles/13/2457/2019/>.
- [10] Erland M Schulson. “Brittle failure of ice”. In: *Engineering Fracture Mechanics* 68.17-18 (2001), pp. 1839–1887. DOI: [10.1016/S0013-7944\(01\)00029-0](https://doi.org/10.1016/S0013-7944(01)00029-0). URL: [https://doi.org/10.1016/S0013-7944\(01\)00029-0](https://doi.org/10.1016/S0013-7944(01)00029-0).
- [9] Gunnar Spreen, Ron Kwok, and Dimitris Menemenlis. “Trends in Arctic sea ice drift and role of wind forcing: 1992–2009”. In: *Geophysical Research Letters* 38.19 (2011). DOI: [10.1029/2011GL048970](https://doi.org/10.1029/2011GL048970). URL: <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2011GL048970>.
- [5] Michel Tsamados et al. “Impact of variable atmospheric and oceanic drag coefficients in a sea ice model”. In: *Journal of Physical Oceanography* 44.5 (2014), pp. 1230–1253. DOI: [10.1175/JPO-D-13-0215.1](https://doi.org/10.1175/JPO-D-13-0215.1). URL: <https://journals.ametsoc.org/view/journals/phoc/44/5/jpo-d-13-0215.1.xml>.

Appendix

Appendix: Python Code

Listing 1: Sensitivity and convergence study for spatial step size ds_2

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # 1. COMMON PHYSICAL PARAMETERS
6 # =====
7 phi = np.radians(15)
8 lambda_angle = np.pi/4 + phi/2      # 52.5 degrees
9 theta_droite = 0.0
10 a_vent = 150.0                       # Wind strength
11
12 # Total physical distance to cover for comparison (e.g., 5 meters)
13 L_totale = 5.0
14
15 # 10 log-spaced ds2 values between 1.0 and 0.0001
16 ds2_values = np.geomspace(1.0, 0.0001, 10)
17
18 liste_Z_finaux = []
19 liste_Y_finaux = []
20
21 # =====
22 # 2. SWEEP OVER DIFFERENT ds2 VALUES
23 # =====
24 for ds2 in ds2_values:
25
26     # Calculation of adapted number of iterations (e.g., 5 to 50,000)
27     max_iterations = int(L_totale / ds2)
28
29     # Reset initial state for each simulation
30     Z_actuel = 110
31     x_P1, y_P1 = 0.0, 0.0
32     theta_P1 = theta_droite + 2 * lambda_angle
33     dTheta = 0.0
34
35     # Propagation loop with original algorithm
36     for i in range(max_iterations):
37         angle_relatif_vent = theta_P1 - (0.0 + np.pi/2)
38         a2_local = a_vent * np.cos(angle_relatif_vent)
39
40         # Differential evolution of Z
41         dZ = (2.0 * np.tan(phi) * Z_actuel * dTheta) + (np.sin(phi) * a2_local * ds2)
42         Z_nouveau = Z_actuel + dZ
43
44         # Calculation of curvature (Kappa)
45         kappa = (a2_local / (4 * max(Z_nouveau, 1e-3))) * np.cos(phi)
46
47         # Recalculation of angle and coordinates
48         theta_P2 = theta_P1 + kappa * ds2
49         x_P2 = x_P1 + ds2 * np.cos(theta_P2)
50         y_P2 = y_P1 + ds2 * np.sin(theta_P2)
51
52         # Update variables
53         x_P1, y_P1 = x_P2, y_P2
54         dTheta = theta_P2 - theta_P1
55         theta_P1 = theta_P2
56         Z_actuel = Z_nouveau
57
58     # Storage of final value reached at end of trajectory
59     liste_Z_finaux.append(Z_actuel)
60     liste_Y_finaux.append(y_P1)
61
62 # =====
63 # 3. PLOT OF Z AND Y EVOLUTION AS A FUNCTION OF ds2
64 # =====
65 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
66
67 # Plot 1: Final shear Z vs ds2 (Log Scale)
68 ax1.plot(ds2_values, liste_Z_finaux, 'b-o', linewidth=2, markersize=6)
69 ax1.set_xscale('log')
```

```

70 ax1.set_xlabel('Spatial_step_{$ds_2$}(meters)_-[Log_scale]')
71 ax1.set_ylabel('Final_shear_{$Z$}')
72 ax1.set_title('Sensitivity_of_final_shear_{$Z$}_to_step_{$ds_2$}')
73 ax1.invert_xaxis() # Display from ds2=1 to ds2=0.0001
74 ax1.grid(True, which='both', linestyle=':', alpha=0.6)
75
76 # Plot 2: Final vertical position Y vs ds2 (Log Scale)
77 ax2.plot(ds2_values, liste_Y_finaux, 'r-s', linewidth=2, markersize=6)
78 ax2.set_xscale('log')
79 ax2.set_xlabel('Spatial_step_{$ds_2$}(meters)_-[Log_scale]')
80 ax2.set_ylabel('Final_height_{$Y$}(meters)')
81 ax2.set_title('Sensitivity_of_final_height_{$Y$}_to_step_{$ds_2$}')
82 ax2.invert_xaxis()
83 ax2.grid(True, which='both', linestyle=':', alpha=0.6)
84
85 plt.tight_layout()
86 plt.savefig('/home/anaelle/Works_2026/anaelle/images/convergences_dsZ10.png')
87 plt.show()
88
89 # Console display of results
90 h1, h2, h3, h4 = 'ds2(m)', 'Iterations', 'Final_Z', 'Final_Y(m)'
91 print(f'{h1:<12}|{h2:<12}|{h3:<12}|{h4:<12}')
92 print('-' * 55)
93 for ds, Z_f, Y_f in zip(ds2_values, liste_Z_finaux, liste_Y_finaux):
94     n_iter = int(L_totale / ds)
95     print(f'{ds:<12.5f}|{n_iter:<12d}|{Z_f:<12.2f}|{Y_f:<12.2f}')

```

Listing 2: Convergence rate analysis and order p calculation

```

1 # Calculation of the absolute error relative to Z_ref = 330.10
2 erreur_Z = abs(Z_actuel - Z_ref)
3
4 liste_Z_finaux.append(Z_actuel)
5 liste_N.append(max_iterations)
6 liste_erreurs.append(erreur_Z)
7
8 # Conversion to NumPy arrays for log calculations
9 N_array = np.array(liste_N)
10 E_array = np.array(liste_erreurs)
11
12 # Filtering to avoid log(0) if error becomes zero
13 valid_mask = E_array > 0
14 N_valid = N_array[valid_mask]
15 E_valid = E_array[valid_mask]
16
17 # Linear regression on log10(E) vs log10(N) to obtain the slope (order p)
18 log_N = np.log10(N_valid)
19 log_E = np.log10(E_valid)
20 pente, ordonnee = np.polyfit(log_N, log_E, 1)
21
22 # =====
23 # 3. PLOT OF LOG(E) VERSUS LOG(N)
24 # =====
25 plt.figure(figsize=(8, 6))
26
27 # Plot of calculated points
28 plt.loglog(N_array, E_array, 'bo-', linewidth=2, markersize=7, label=r'Error_{$E$}_|Z_-$Z_{\mathrm{ref}}$|')
29
30 # Plot of the trend / regression line
31 plt.loglog(N_valid, 10**(ordonnee) * N_valid**(pente), 'r--',
32            label=f'Slope_(order_of_convergence)_={pente:.2f}')
33
34 plt.xlabel('Number_of_iterations_{$N$}_[Log_Scale]', fontsize=11)
35 plt.ylabel(r'Shear_error_{$E$}_|Z_-$Z_{\mathrm{final}}$|_330.10|$_[Log_Scale]', fontsize=11)
36 plt.title(r'Convergence_analysis: $\log(E)$ versus $\log(N)$', fontsize=12)
37 plt.grid(True, which='both', linestyle=':', alpha=0.6)
38 plt.legend(fontsize=10)
39
40 plt.tight_layout()
41 plt.savefig('/home/anaelle/Works_2026/anaelle/images/methode1_logE.png')
42 plt.show()
43

```

```

44 # Console output of results with higher precision (8 decimals)
45 h1, h2, h3, h4 = 'ds2(m)', 'N(iterations)', 'Final_Z', 'Error_E'
46 print(f'{h1:<15}|{h2:<15}|{h3:<18}|{h4:<15}')
47 print('-' * 70)
48
49 for ds, n, Z_f, err in zip(ds2_values, N_array, liste_Z_finaux, E_array):
50     # .8f = 8 decimal places / .6e = scientific notation with 6 decimals
51     print(f'{ds:<15.8f}|{n:<15d}|{Z_f:<18.8f}|{err:<15.6e}')
52
53 print(f'\n--->Measured_slope(scheme_order):{pente:.6f}')

```

Listing 3: S3 computing program

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3     from scipy.interpolate import UnivariateSpline
4     from scipy.optimize import brentq
5     from matplotlib.collections import LineCollection
6
7
8     # 1. PARAM TRES
9
10    phi = np.radians(15)
11    lambda_angle = np.pi/4 + phi/2
12    ds1 = 1.0
13    N_s1 = 100
14    ds2 = 0.005
15    max_iter_s2 = 200000
16    tolerance_Z = 1e-5
17    a_vent = 150.0
18    N_fences = 2 # 2 Fences = 200 branches total (100 par fence)
19
20    all_branches_x, all_branches_y, all_branches_shear = [], [], []
21    all_branch_splines = []
22
23    x_fissure_actuelle = np.linspace(-100.0, 100.0, N_s1)
24    y_fissure_actuelle = np.zeros(N_s1)
25    Z_max_etage_precedent = 100.0
26
27
28    # 2. G N RATION DES FENCES (S2)
29
30    for k in range(N_fences):
31        x_prochaine_fence, y_prochaine_fence = [], []
32        Z_finaux_branche = []
33        Z_actuel = Z_max_etage_precedent
34
35        for i in range(N_s1):
36            x_P1, y_P1 = x_fissure_actuelle[i], y_fissure_actuelle[i]
37            theta_P1 = 0.0 + 2 * lambda_angle
38            dTheta, Z_actuel = 0.0, Z_max_etage_precedent
39
40            x_s2, y_s2, Z_s2, theta_s2, shear_s2 = [x_P1], [y_P1], [Z_actuel], [theta_P1],
41                [0.0]
42
43            j = 0
44            diff_Z = 1.0
45            while diff_Z >= tolerance_Z and j < max_iter_s2:
46                a2_local = a_vent * np.cos(theta_P1 - np.pi/2)
47                decay_factor = np.exp(-2.0 * np.tan(phi) * np.abs(dTheta))
48                Z_nouveau = Z_actuel * decay_factor + (np.sin(phi) * a2_local * ds2)
49                Z_secu = max(Z_nouveau, 1e-3)
50                kappa = (a2_local / (4 * Z_secu)) * np.cos(phi)
51
52                theta_P2 = theta_P1 + kappa * ds2
53                x_P2 = x_P1 + ds2 * np.cos(theta_P2)
54                y_P2 = y_P1 + ds2 * np.sin(theta_P2)
55
56                x_s2.append(x_P2); y_s2.append(y_P2); Z_s2.append(Z_nouveau); theta_s2.
57                    append(theta_P2); shear_s2.append(a2_local)
58                diff_Z = abs(Z_nouveau - Z_actuel)
59                x_P1, y_P1, dTheta, theta_P1, Z_actuel = x_P2, y_P2, (theta_P2 - theta_P1),
60                    theta_P2, Z_nouveau

```

```

58         j += 1
59
60         min_len = min(len(x_s2), len(y_s2), len(Z_s2), len(shear_s2))
61         x_s2, y_s2, Z_s2, shear_s2 = x_s2[:min_len], y_s2[:min_len], Z_s2[:min_len],
62         shear_s2[:min_len]
63         all_branches_x.append(x_s2); all_branches_y.append(y_s2); all_branches_shear.
64         append(shear_s2)
65
66         t = np.linspace(0, 1, min_len)
67         all_branch_splines.append({'x': UnivariateSpline(t, x_s2, s=0.00001), 'y':
68         UnivariateSpline(t, y_s2, s=0.00001), 'Z': UnivariateSpline(t, Z_s2, s
69         =0.00001)})
70         x_prochaine_fence.append(x_s2[-1]); y_prochaine_fence.append(y_s2[-1]);
71         Z_finaux_branche.append(Z_nouveau)
72
73         x_fissure_actuelle = np.linspace(-100.0, 100.0, N_s1)
74         idx_sort = np.argsort(x_prochaine_fence)
75         y_fissure_actuelle = np.interp(x_fissure_actuelle, np.array(x_prochaine_fence)[
76         idx_sort], np.array(y_prochaine_fence)[idx_sort])
77         Z_max_etage_precedent = np.max(Z_finaux_branche)
78
79 # 3 G N RATEUR DE CHA NE
80
81 def solve_intersection_system(idx_dest, x1, y1, angle):
82     s_x, s_y = all_branch_splines[idx_dest]['x'], all_branch_splines[idx_dest]['y']
83     def system_eq(t): return (s_y(t) - y1) * np.cos(angle) - (s_x(t) - x1) * np.sin(
84     angle)
85     t_vals = np.linspace(0.001, 0.999, 1000)
86     f_vals = [system_eq(t) for t in t_vals]
87     for k in range(len(t_vals)-1):
88         if f_vals[k] * f_vals[k+1] <= 0:
89             try:
90                 t_sol = brentq(system_eq, t_vals[k], t_vals[k+1], xtol=1e-14)
91                 x_sol, y_sol = float(s_x(t_sol)), float(s_y(t_sol))
92                 k_dist = (x_sol - x1)/np.cos(angle) if abs(np.cos(angle)) > 1e-3 else (
93                 y_sol - y1)/np.sin(angle)
94                 if k_dist > -1e-3: return t_sol, (x_sol, y_sol)
95             except: continue
96     return None, None
97
98 def generate_chain(start_idx, start_t):
99     data = []
100     current_idx, current_t = start_idx, start_t
101     %On limite 100 pas max pour ne pas boucler ind finiment
102     for _ in range(100):
103         if current_idx >= len(all_branch_splines) - 1: break
104         s_x_cur, s_y_cur = all_branch_splines[current_idx]['x'], all_branch_splines[
105         current_idx]['y']
106         dx, dy = s_x_cur.derivative()(current_t), s_y_cur.derivative()(current_t)
107         theta_jump = np.arctan2(dy, dx) - 2 * lambda_angle
108         x_deb, y_deb = float(s_x_cur(current_t)), float(s_y_cur(current_t))
109         t_impact, pt_fin = solve_intersection_system(current_idx + 1, x_deb, y_deb,
110         theta_jump)
111         if pt_fin is None: break
112         data.append({'p1': (x_deb, y_deb), 'p2': pt_fin, 'Z': float(all_branch_splines[
113         current_idx+1]['Z'](t_impact))})
114         current_idx, current_t = current_idx + 1, t_impact
115     return data
116
117 % G n ration des deux cha nes
118 data_chaine_1 = generate_chain(0, 0.005) % Cha ne 1 (Fence 1)
119 data_chaine_2 = generate_chain(100, 0.005) % Cha ne 2 (Fence 2, d bute 1'index_100)
120
121 # 4 VISUALISATION COMBIN E
122
123 all_shear_flat = np.concatenate([np.array(s).ravel() for s in all_branches_shear])
124 norm = plt.Normalize(vmin=all_shear_flat.min(), vmax=all_shear_flat.max())
125 cmap = plt.cm.viridis
126
127 plt.figure(figsize=(20, 8))
128 for i in range(len(all_branches_x)):

```

```

120 shear_vals=np.array(all_branches_shear[i]).ravel()
121 points=np.array([all_branches_x[i],all_branches_y[i]]).T.reshape(-1,1,2)
122 segments=np.concatenate([points[:-1],points[1:]],axis=1)
123 lc=LineCollection(segments,cmap=cmap,norm=norm,alpha=0.3,linewidth=0.8)
124 lc.set_array(shear_vals)
125 plt.gca().add_collection(lc)
126
127 %Tracer_Chaine1(Rouge,plein)
128 for item in data_chaine_1:
129     plt.plot([item['p1'][0],item['p2'][0]], [item['p1'][1],item['p2'][1]], 'r.-',
130             linewidth=2,zorder=10)
131
132 %Tracer_Chaine2(Bleu,fonc ,pointill )
133 for item in data_chaine_2:
134     plt.plot([item['p1'][0],item['p2'][0]], [item['p1'][1],item['p2'][1]], 'b.--',
135             linewidth=2,zorder=10)
136
137 plt.title( Comparison_of_Fracture_Chains: Fence1(Red)vsFence2(Blue) )
138 plt.xlabel( X_Position );plt.ylabel( Y_Position )
139 plt.axis('equal');plt.grid(True,linestyle=':',alpha=0.6)
140 plt.colorbar(plt.cm.ScalarMappable(norm=norm,cmap=cmap),ax=plt.gca(),label='Shear
141 Stress Intensity')
142
143 save_path='/home/anaelle/Works_2026/anaelle/images/fracture_chain_comparison.png'
144 plt.savefig(save_path,dpi=300,bbbox_inches='tight')
145 plt.show()

```

Listing 4: lkf propagation program

```

1 import numpy as np
2 import pandas as pd
3 import xarray as xr
4 from scipy.interpolate import splprep, splev
5 import matplotlib.pyplot as plt
6 import time
7
8 t_start = time.time()
9
10 # =====
11 # 1. PARAM TRES ET CONFIGURATION
12 # =====
13 phi = np.radians(15)
14 lambda_angle = np.pi / 4 + phi / 2
15 ds2 = 0.005
16 tol_dZ = 1e-1 # Remis 1e-1 pour converger plus vite ( vite les boucles trop longues )
17 max_iter_s2 = 1000 # R duit 1000 pour viter de bloquer inutilement
18 N_fences = 1
19 target_id = 10553
20
21 # =====
22 # 2. PR -TRAITEMENT LKF
23 # =====
24 df_target = data_filtree[data_filtree['LKF_No.'] == target_id].copy()
25 mask = (df_target['lat'].diff() != 0) | (df_target['lon'].diff() != 0)
26 df_target = df_target[mask]
27
28 lat0, lon0 = df_target['lat'].iloc[0], df_target['lon'].iloc[0]
29 x_m_raw = (df_target['lon'].values - lon0) * 111000 * np.cos(np.radians(lat0))
30 y_m_raw = (df_target['lat'].values - lat0) * 111000
31 shear_raw = df_target['shear_rate'].values
32
33 s_factor = len(x_m_raw) * 10.0
34 tck, u = splprep([df_target['lon'].values, df_target['lat'].values, x_m_raw, y_m_raw], s
35                 =s_factor, k=3)
36
37 u_fine = np.linspace(0, 0.2, 10)
38 lon_lkf, lat_lkf, x_lkf, y_lkf = splev(u_fine, tck)
39 shear_lkf = np.interp(u_fine, u, shear_raw) * 1000
40
41 _, _, dx, dy = splev(u_fine, tck, der=1)
42 theta_lkf = np.arctan2(dy, dx)

```

```

43 # =====
44 # 3. PARATION DU VENT (UNE SEULE FOIS)
45 # =====
46 ds_window = ds_wind.sel(time=slice( 2007-04-02 , 2007-04-04 ))
47 ds_wind_mean = ds_window.mean(dim='time')
48
49 lon_name = 'longitude' if 'longitude' in ds_wind_mean.coords else 'lon'
50 lat_name = 'latitude' if 'latitude' in ds_wind_mean.coords else 'lat'
51
52 d_min_lon, d_max_lon = ds_wind_mean[lon_name].min().values, ds_wind_mean[lon_name].max()
53 d_min_lat, d_max_lat = ds_wind_mean[lat_name].min().values, ds_wind_mean[lat_name].max()
54
55 # Clamping de s curit des points de la LKF
56 lon_lkf_safe = np.clip(lon_lkf, d_min_lon + 0.1, d_max_lon - 0.1)
57 lat_lkf_safe = np.clip(lat_lkf, d_min_lat + 0.1, d_max_lat - 0.1)
58
59 # Interpolation vectoris e unique pour toute la LKF initiale
60 coords_da_lon = xr.DataArray(lon_lkf_safe, dims= points )
61 coords_da_lat = xr.DataArray(lat_lkf_safe, dims= points )
62 wind_precomputed = ds_wind_mean.interp({lon_name: coords_da_lon, lat_name: coords_da_lat
63 }, method= linear )
64
65 u_lkf_vals = np.nan_to_num(wind_precomputed['eastward_wind'].values, nan=0.0)
66 v_lkf_vals = np.nan_to_num(wind_precomputed['northward_wind'].values, nan=0.0)
67
68 print( Pr -calcul du vent termin . )
69
70 # =====
71 # 4. SIMULATION DE PROPAGATION
72 # =====
73 fig, (ax_global, ax_zoom, ax_z) = plt.subplots(3, 1, figsize=(10, 15), gridspec_kw={'
74 height_ratios': [2, 2, 1]})
75 cmap = plt.cm.tab10(np.linspace(0, 1, N_fences))
76
77 x_curr, y_curr = x_lkf.copy(), y_lkf.copy()
78 theta_curr = theta_lkf.copy()
79 Z_curr = shear_lkf.copy()
80 d_theta_arr = np.append(np.diff(theta_curr), 0)
81
82 first_s2_x, first_s2_y, first_s2_z = [], [], []
83
84 for k in range(N_fences):
85     x_next, y_next, theta_next, Z_next = [], [], [], []
86
87     for i in range(len(x_curr)):
88         x_P1, y_P1 = x_curr[i], y_curr[i]
89         theta_P1 = theta_curr[i] + lambda_angle
90         Z_act = Z_curr[i]
91         d_theta = d_theta_arr[i]
92
93         # Utilisation directe du vent pr -calcul pour ce point
94         u_w = u_lkf_vals[i]
95         v_w = v_lkf_vals[i]
96
97         j = 0
98         dZ = float('inf')
99
100         while abs(dZ) > tol_dZ and j < max_iter_s2:
101             v_proj = u_w * np.cos(theta_P1) + v_w * np.sin(theta_P1)
102             a2 = np.maximum(1e-4, v_proj)
103
104             decay = 2.0 * np.tan(phi) * d_theta
105             Z_new = Z_act * (1 + decay) + (np.sin(phi) * a2 * ds2)
106             Z_new = np.clip(Z_new, 1e-4, 50.0)
107
108             dZ = Z_new - Z_act
109
110             # Formule de kappa corrig e
111             kappa = (a2 / (4.0 * np.maximum(Z_new, 1e-2))) * np.cos(phi)
112             kappa = np.clip(kappa, -0.05, 0.05)

```

```

112     theta_P2 = theta_P1 + kappa * ds2
113     x_P2 = x_P1 + ds2 * np.cos(theta_P2)
114     y_P2 = y_P1 + ds2 * np.sin(theta_P2)
115
116     d_theta = theta_P2 - theta_P1
117
118     # Enregistrement pour le premier point (i=0, k=0)
119     if i == 0 and k == 0:
120         if j == 0:
121             first_s2_x = [x_P1]
122             first_s2_y = [y_P1]
123             first_s2_z = [Z_act]
124             first_s2_x.append(x_P2)
125             first_s2_y.append(y_P2)
126             first_s2_z.append(Z_new)
127
128     x_P1, y_P1, theta_P1, Z_act = x_P2, y_P2, theta_P2, Z_new
129     j += 1
130
131     x_next.append(x_P1); y_next.append(y_P1)
132     theta_next.append(theta_P1); Z_next.append(Z_act)
133
134     x_curr, y_curr, theta_curr, Z_curr = np.array(x_next), np.array(y_next), np.array(
135         theta_next), np.array(Z_next)
136     ax_global.plot(x_curr, y_curr, '-', color=cmap[k], alpha=0.5)
137     ax_zoom.plot(x_curr, y_curr, '-', color=cmap[k], alpha=0.7)
138
139     # Trac de Z sur ax_z
140     if len(first_s2_z) > 0:
141         ax_z.plot(range(len(first_s2_z)), first_s2_z, 'r.-', label='volution de Z(i=0)')
142         ax_z.set_title('volution de Z au cours des it rations de la premi re propagation s2')
143         ax_z.set_xlabel('Nombre d\'it rations(j)')
144         ax_z.set_ylabel('Valeur de Z')
145         ax_z.grid(True)
146         ax_z.legend()
147
148     # =====
149     # 5. FINALISATION
150     # =====
151     ax_global.set_title(f'Propagation LKF_{target_id}')
152     ax_global.axis('equal')
153     ax_global.grid(True)
154
155     margin = 500
156     if len(x_curr) > 0 and np.isfinite(x_curr).all():
157         ax_zoom.set_xlim(np.min(x_curr) - margin, np.max(x_curr) + margin)
158         ax_zoom.set_ylim(np.min(y_curr) - margin, np.max(y_curr) + margin)
159
160     ax_zoom.set_title('Zoom Propagation globale')
161     ax_zoom.grid(True)
162     plt.tight_layout()
163     plt.show()
164
165     print(f Temps d'ex cution total : {time.time() - t_start:.2f} secondes )

```

Explication des cotations	
I	(I) Insuffisant
S	(S) Suffisant
B	(B) Bien
E	(E) Excellent
N	(N) Non applicable

Fiche d'évaluation

étudiant

Ce document est à insérer dans le rapport. Le rapport est à fournir pour le 24 aout 2026 dernier délai.

Organisme d'accueil : Sujet
du stage :

Evaluation qualitative du stage	I	S	B	E	N	Commentaires
Qualité de l'entreprise						
L'entreprise offre-t-elle un contexte propice à une carrière d'ingénieur ?					X	
L'entreprise connaissait elle l'école (accueil de stagiaire, embauche, relations autres...) ?				X		
L'entreprise a-t-elle mis à votre dispositions les moyens nécessaires pour réaliser votre mission (documents, éléments d'information, matériels) ?			X			
Qualité de la missions						
Vos missions étaient-elles en rapport avec votre formation ?			X			
Les missions effectuées étaient-elles bien celles définies au départ ?			X			
Qualité de l'encadrement						
Votre tuteur organisme d'accueil a-t-il pris le temps de vous présenter le fonctionnement de la structure et l'équipe ?		X				
Votre tuteur organisme d'accueil vous a-t-il aidé et conseillé quand cela était nécessaire ?			X			
Votre enseignant référent vous a-t-il aidé et conseillé lorsque cela était nécessaire ?					X	

NB : Dans le but d'alléger la lecture du document, le genre masculin est utilisé sans discrimination pour le genre masculin et féminin.

Explication des cotations	
I	(I) Insuffisant
S	(S) Suffisant
B	(B) Bien
E	(E) Excellent
N	(N) Non applicable

Fiche d'évaluation

étudiant

Autoévaluation : développement des compétences et trajectoire professionnelle

En prenant un peu de recul sur votre activité durant le stage pensez-vous avoir travaillé / développé certaines des compétences du référentiel de la formation Seatech ? Lesquelles ? Pourquoi et comment ? D'autres compétences ?

Observations sur les critères :

Les critères doivent être développés, même partiellement, au cours du stage de 2A.

Utiliser, autant que possible, les mises en situation du stage de 2A pour consolider ces critères déjà développés en stage 1A

Compétence	Critère	Commentaire
Concevoir des solutions d'ingénierie	Analyser le besoin	Analyse du besoin opérationnel de la navigation polaire : anticiper le mouvement de la glace (ouvertures d'eau libre vs zones de pression destructrices) pour sécuriser et optimiser la route des navires. Ce besoin a été traduit techniquement par le développement d'un outil d'analyse mésoéchelle mesurant la déformation (LKF, shear) de la banquise à partir de données satellites (RGPS).
	Répondre au besoin	Développement d'un algorithme Python exploitant les données satellitaires RGPS pour calculer quantitativement le taux de cisaillement (<i>shear rate</i>) et suivre la propagation des fractures (LKF). En appliquant la Méthode des Caractéristiques (MoC), l'outil reconstruit automatiquement le profil mécanique continu le long des fissures pour caractériser l'état de déformation de la glace.
	Utiliser les outils appropriés	Manipulation de l'environnement de développement Emacs et d'outils d'analyse interactive (Jupyter) pour concevoir des scripts Python. Ces outils ont permis le traitement automatisé, la modélisation mathématique et la visualisation graphique des jeux de données géophysiques complexes RGPS.
	Documenter les choix, les sources et les références	Réalisation d'une recherche bibliographique approfondie en m'appuyant directement sur les publications scientifiques de mon tuteur de stage. Cette étude de l'état de l'art a permis de justifier les choix théoriques retenus (équations d'invariants, angle de frottement ϕ , géométrie des caractéristiques) et d'assurer la validité scientifique de l'algorithme développé.
Mettre en œuvre des solutions	Analyser et améliorer une solution existante	Analyse critique des travaux antérieurs et étude approfondie de la méthode développée par Björn Erlingsson à travers l'examen des rapports de stage et publications précédents. L'assimilation de ces travaux existants m'a permis d'identifier les limites de leurs implémentations et d'améliorer la solution (optimisation du calcul des caractéristiques, correction de la trajectoire et gestion des sauts géométriques dans le code).
Développer une démarche R&D	Réaliser une veille technologique / un état de l'art	Réalisation d'une veille scientifique et technologique sur l'analyse des fractures de la banquise (LKF) et le traitement des données RGPS
	Formuler des hypothèses	Formulation d'hypothèses physiques clés pour la modélisation : prise en compte du vent comme force motrice principale de la dérive de la banquise et intégration de l'angle de friction interne de la glace (ϕ) pour régir la géométrie de propagation des ruptures (angles λ).

Explication des cotations	
I	(I) Insuffisant
S	(S) Suffisant
B	(B) Bien
E	(E) Excellent
N	(N) Non applicable

Fiche d'évaluation

étudiant

Piloter des projets d'ingénierie	S'inscrire dans le pilotage d'un projet	Travail en binôme/groupe : répartition claire des tâches de développement, mise en commun et validation croisée de nos résultats scientifiques, et entraide régulière pour assimiler les concepts théoriques complexes du sujet.
	Prendre en compte la gestion globale des organisations et les règles économiques et juridiques	
	Utiliser les outils de gestion de projets et collaboratifs	Travail en binôme/groupe : répartition claire des tâches de développement, mise en commun et validation croisée de nos résultats scientifiques, et entraide régulière pour assimiler les concepts théoriques complexes du sujet.

Explication des cotations	
I	(I) Insuffisant
S	(S) Suffisant
B	(B) Bien
E	(E) Excellent
N	(N) Non applicable

Fiche d'évaluation

étudiant

Encadrer une équipe	S'insérer et collaborer	
	Travailler en équipe pluridisciplinaire et/ou internationale	Intégration au sein d'une structure universitaire internationale (Université des Westfjords), nécessitant une collaboration en anglais avec des profils scientifiques variés et une adaptation à une culture de travail différente.
	Communiquer (écrit et oral) de manière adaptée	Restitution régulière de l'avancée des travaux à l'écrit et à l'oral grâce à des réunions ou des mini rapport.
Agir en professionnel responsable	Prendre en compte les enjeux éthiques et sociétaux (RSE, DD, RGPD, ...)	Prise en compte directe des enjeux du changement climatique en Arctique : la modélisation de la dérive des glaces vise à sécuriser la navigation polaire et à prévenir les risques d'accidents environnementaux (marées noires) en zone fragile.
	Assumer la responsabilité de ses actes et décision	Autonomie et prise d'initiative pour débloquer le sujet malgré le manque de directives au démarrage, en faisant des choix algorithmiques assumés et en vérifiant systématiquement la cohérence physique de mes résultats.
	Être dans une dynamique d'apprentissage	Capacité d'auto-formation rapide pour maîtriser l'environnement Linux/Emacs, l'analyse de données RGPS et la méthode des caractéristiques, transformant un manque initial d'informations en une opportunité de montée en compétences.

Trajectoire professionnelle

A la suite de votre stage, avez-vous confirmé ou affiné votre projet professionnel d'être ingénieur (métier plus précis, secteur, contexte ou type d'entreprise, ...) ? Si oui, quelles actions pensez-vous devoir entreprendre pour y arriver (renforcer certaines connaissances, développer certaines compétences, lesquelles)?

Oui, ce stage m'a conforté dans mon choix de devenir ingénieur en calculs dans le milieu naval. Pour y parvenir, mes principales actions seront de renforcer ma communication orale, d'apprendre à mieux m'imposer en groupe et de donner plus facilement mon avis au sein d'une équipe.

Date et signature de l'étudiant :

10/08/2026

